```
    };
};

export default Home;
```

Listing 14-6: The pages/index.tsx *file*

We implemented the Next.js page, similar to the structure discussed in Chapter 5. First we import all dependencies; then we create the `NextPage` and store it in a constant that we export at the end of the file ❶.

The Next.js page's `props` object, the page properties, contains the data we return from the `getStaticProps` function ❺, discussed in Chapter 5. In this asynchronous function, we connect to the database ❸. As soon as the connection is ready, we call the service method to retrieve all locations ❹ and then pass them as a JSON string to the `NextPage` in the `data.locations` property of the `props` object. Next.js calls the `getStaticProps` function on build time and generates the HTML for this page only once. We can use this rendering method because the list of available locations never changes; it is static.

Then we retrieve the locations from the page properties ❷, parse the JSON string back to an array, and store the page title in a variable. We type the `locations` constant explicitly because TSC cannot easily infer the type. Then we construct the JSX. In the first step, we use the `next/head` component to set the page-specific metadata. Then we call the `LocationList` component we previously implemented with the `locations` array in the `locations` attribute. By doing so, the `LocationList` component renders all locations as an overview list.

As soon as you save the file, you should see, in the Docker command line, that Next.js recompiles the application. Open the web application at *http://localhost:3000* in your browser to see a list of locations similar to Figure 14-1.

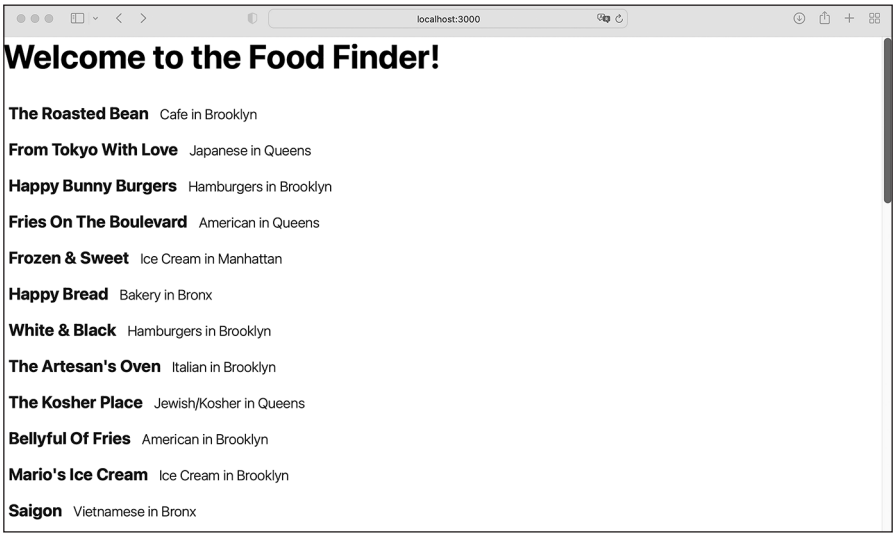


Figure 14-1: The start page showing all available locations

Now we'll move on to styling the frontend and adding basic global components, such as the application's header with the Food Finder logo.

The Global Layout Components

Now it's time to create the three global components. These include the overall layout component, which we'll use to format the start and wish list page content, a *sticky* header (which is always visible, "sticking" to the browser's upper edge), and the Food Finder logo to go in the header. Again, we'll start with the smallest units and then use those as building blocks for the overall components.

The Logo

The smallest component, the logo, is nothing more than a `next/image` component wrapped in a `next/link` element; when users click the logo image, they'll be redirected to the start page. Add a *header* folder to the *components* folder, then add a *logo* folder to the *header* folder and create two files there, *index.tsx* and *index.module.css*, into which you should paste the code in Listing 14-7.

```
.root {
    display: inline-block;
    height: 35px;
    position: relative;
    width: 119px;
}

@media (min-width: 600px) {
    .root {
        height: 50px;
        width: 169px;
    }
}
```

Listing 14-7: The components/header/logo/index.module.css *file*

These basic styles for the component's `root` element set the image's dimensions. We use a *mobile-first design pattern* by initially defining the styles to use on smaller screens and then, using a standard CSS media query, modifying them for screens bigger than 600px. We'll use a bigger image on bigger screens.

Now let's create the logo component. Create an *assets* subfolder in the Next.js *public* folder and place the *logo.svg* file extracted from *assets.zip* into it. Then add the code in Listing 14-8 to the logo's *index.tsx* file.

```
import Image from "next/image";
import Link from "next/link";
import logo from "/public/assets/logo.svg";
import styles from "./index.module.css";
```

```
const Logo = (): JSX.Element => {
    return (
        <Link href="/" passHref className={styles.root}>
            <Image
                src={logo}
                alt="Logo: Food Finder"
                sizes="100vw"
                fill
                priority
            />
        </Link>
    );
};

export default Logo;
```

Listing 14-8: The components/header/logo/index.tsx *file*

As usual, we import the dependencies and then create an exported constant that contains the JSX code. We don't pass any data to it through attributes or child elements; hence, we don't need to define the component's `props` object here.

We use a basic `next/image` inside a `next/link` element to link back to the start page and set the `next/image`'s attributes to fill the available space defined in the CSS file.

The Header

The header component will wrap the logo component we just created. Create the *index.tsx* file and *index.module.css* file in the *header* folder, then add the code in Listing 14-9 to the CSS file.

```
.root {
    background: white;
    border-bottom: 1px solid #eaeaea;
    padding: 1rem 0;
    position: sticky;
    top: 0;
    width: 100%;
    z-index: 1;
}
```

Listing 14-9: The components/header/index.module.css *file*

We use the CSS definitions `position: sticky` and `top: 0` to stick the header to the upper edge of the browser. Now the header will automatically stay there even when users scroll down the page; the page's content should scroll below the header because we set the header's `z-index`, placing the header in front of the other elements. You can think of the `z-index` as determining which floor of a building an element is on.

Listing 14-10 shows the code for the header component. Copy it into the component's *index.tsx* file.

```
import styles from "./index.module.css";
import Logo from "components/header/logo";

const Header = (): JSX.Element => {
    return (
        <header className={styles.root}>
            <div className="layout-grid">
                <Logo />
            </div>
        </header>
    );
};

export default Header;
```

Listing 14-10: The components/header/index.tsx *file*

We define a basic component that displays the logo. Then we wrap the imported `Logo` component in an element with a global `layout-grid` class, which we'll define in the next section.

The Layout

Currently, we have one Next.js page (the start page) and a header component. The easiest way to add the header to the page would be to import it into the Next.js page and place it directly into the JSX. However, we'll add two more pages to the app, the wish list page and the location detail page, so we want to avoid importing the header three times.

To streamline the overall app design, Next.js provides the concept of a *layout*, which is really just another component, and we can use it to add the header component as a sibling element to a page's content. Let's create a new layout component. First, to create this component's CSS file, add *layout.css* to the *styles* folder and paste the code in Listing 14-11 into it.

```
.layout-grid {
    align-items: center;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    margin: 0 auto;
    max-width: 800px;
    padding: 0 1rem;
    width: 100%;
}

@media (min-width: 600px) {
    .layout-grid {
        flex-direction: row;
        padding: 0 2rem;
    }
}
```

Listing 14-11: The styles/layout.css *file*

We use the mobile-first pattern once again to define a basic grid wrapper, setting the global padding and maximum width for the content area. We set the wrapper's left and right `margins` to `auto`, which centers the container, because the margins take up all available space between the fixed-width wrapper and the window's edges.

We use `flexbox` to set the direction of the wrapper's direct child elements to `column`, displaying them one on top of the next. Because the logo and all other upcoming header elements are direct children of an element with the `layout-grid` class, they are affected by the `flexbox` layout. In contrast, the location items aren't direct siblings. Hence, they won't change their direction when switching between screen sizes.

Then we use a media query to adjust the styles for screens whose width is greater than 600px. Here we increase the padding and change the layout order of the direct child elements. Instead of using `column`, we set it to `row`, and immediately we display the elements next to one another.

Because this is a global styles file and not a CSS module, Next.js won't automatically scope the class names. Hence, we prefix them with `layout-` and don't import the styles into the component before using them.

Now create a *layout* folder inside the *components* folder and add the *index.tsx* file to it with the component code in Listing 14-12.

```
import Header from "components/header";

interface PropsInterface {
    children: React.ReactNode;
}

const Layout = (props: PropsInterface): JSX.Element => {
    return (
        <>
            <Header />
            <main className="layout-grid">
                {props.children}
            </main>
        </>
    );
};
export default Layout;
```

Listing 14-12: The components/layout/index.tsx *file*

In the layout component, we define a private interface and the component with the usual structure. Inside the component, we add the `Header` and the `main` element that uses the global layout styles and acts as a wrapper for the `children` elements we'll pass to this component in the *_app.tsx* file.

Open the *_app.tsx* file and modify it as shown in Listing 14-13.

```
import "../styles/globals.css";
import "../styles/layout.css";
import type { AppProps } from "next/app";
import Layout from "components/layout";
```

```
export default function App({ Component, pageProps }: AppProps) {
    return (
        <Layout>
            <Component {...pageProps} />
        </Layout>
    );
}
```

Listing 14-13: The pages/_app.tsx *file*

First we add *layout.css* as a global style. As for the layout, we have only one `layout` component we'll use for all pages, and we import it here. Then we wrap our application, the pages, with the layout and pass the current page in the component's `children` property.

Now all our Next.js pages will follow the same structure: they'll have the `Header` component next to the `main` element containing the page's content. One advantage of following this pattern is that the component's state will be preserved across page changes and React component re-rendering.

Once Next.js has recompiled the application, try reloading the application at *http://localhost:3000* in your browser. It should look like Figure 14-2.

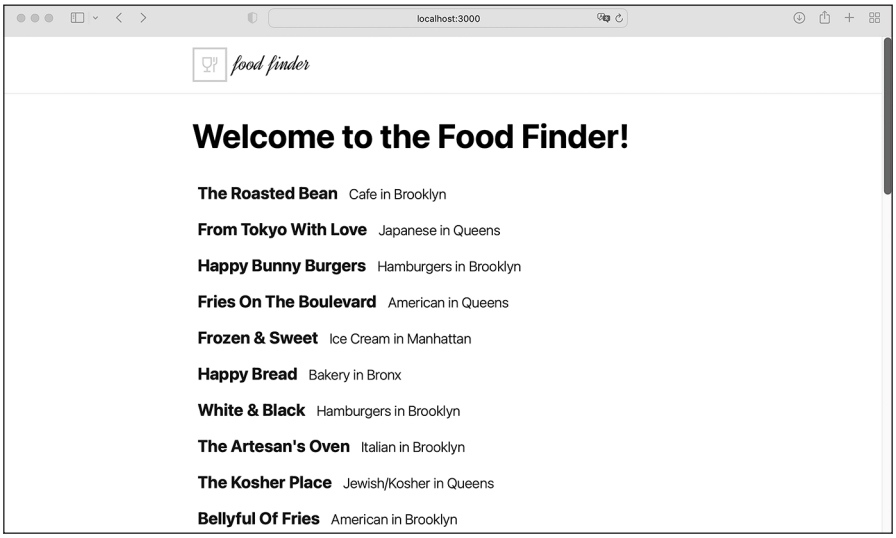


Figure 14-2: The start page with the header and layout component

You should now see the header, and the new layout component centers the content.

The Location Details Page

Our application now has a start page with a header and a list of all available locations. The list items link to their particular location's detail page because we added a `next/link` component to them, but those pages don't exist yet. If you click one of the links, you'll get a *404* error. To display the location details pages, we first need to implement the component that lists a particular location's details and then create a new Next.js page.

The Component

Let's start with the details component. Create the *location-details* folder in the *components* directory and add the *index.module.css* and *index.tsx* files to it. Then add the code from Listing 14-14 to the CSS module.

```
.root {
    margin: 0 0 2rem 0;
    padding: 0;
}
.root li {
    list-style: none;
    margin: 0 0 0.5rem 0;
}
```

Listing 14-14: The components/locations-details/index.module.css *file*

The styles for the component are basic. We remove the default margin and padding, as well as the list styles, and then add a custom margin at the end of each list item and root element.

To implement the location details component, add the code from Listing 14-15 to the *index.tsx* file in the *components/locations-details* folder.

```
import { LocationType } from "mongoose/locations/schema";
import styles from "./index.module.css";

interface PropsInterface {
    location: LocationType;
}

const LocationDetail = (props: PropsInterface): JSX.Element => {
    let location = props.location;
    return (
        <div>
            {location && (
                <ul className={styles.root}>
                    <li>
                        <b>Address: </b>
                        {location.address}
                    </li>
                    <li>
                        <b>Zipcode: </b>
                        {location.zipcode}
                    </li>
```

```
                    <li>
                        <b>Borough: </b>
                        {location.borough}
                    </li>
                    <li>
                        <b>Cuisine: </b>
                        {location.cuisine}
                    </li>
                    <li>
                        <b>Grade: </b>
                        {location.grade}
                    </li>
                </ul>
            )}
        </div>
    );
};
export default LocationDetail;
```

Listing 14-15: The components/locations-details/index.tsx *file*

The locations detail component is structurally similar to the locations list item. Both take an object containing the location's data and add a specific set of properties to the returned JSX element. The main difference is in the JSX structure we create. Otherwise, we follow the known pattern, importing the required styles and type, defining the component's `props` interface using the `LocationType`, and then returning a JSX element with the location details.

The Page

We mentioned in "Overview of the User Interface" on page 215 that a location's detail page should be available at the dynamic URL */location/:location_id*. To implement this, create the *location* folder in the *pages* directory and add the *[locationId].tsx* file containing the code in Listing 14-16.

```
import Head from "next/head";
import type {
    GetServerSideProps,
    GetServerSidePropsContext,
    InferGetServerSidePropsType,
    PreviewData,
    NextPage,
} from "next";
import LocationDetail from "components/location-details";
import dbConnect from "middleware/db-connect";
import { findLocationsById } from "mongoose/locations/services";
import { LocationType } from "mongoose/locations/schema";
import { ParsedUrlQuery } from "querystring";
```

```
const Location: NextPage = (
    props: InferGetServerSidePropsType<typeof getServerSideProps>
) => {
    let location: LocationType = JSON.parse(props.data?.location);
  ❶ let title = `The Food Finder - Details for ${location?.name}`;
    return (
        <div>
            <Head>
                <title>{title}</title>
                <meta
                    name="description"
                    content={`The Food Finder.
                        Details for ${location?.name}`}
                />
            </Head>
            <h1>{location?.name}</h1>
          ❷ <LocationDetail location={location} />
        </div>
    );
};

❸ export const getServerSideProps: GetServerSideProps = async (
    context: GetServerSidePropsContext<ParsedUrlQuery, PreviewData>
) => {
     let locations: LocationType[] | [];
  ❹ let { locationId } = context.query;
     try {
        await dbConnect();
        locations = await findLocationsById([locationId as string]);
      ❺ if (!locations.length) {
            throw new Error(`Locations ${locationId} not found`);
        }
    } catch (err: any) {
        return {
            notFound: true,
        };
    }
    return {
      ❻ props: { data: { location: JSON.stringify(locations.pop()) } },
    };
};
```

Listing 14-16: The pages/location/[locationId].tsx *file*

The start page and location detail page look fairly similar. The only visual difference is the page's title, which we construct with the location's name ❶, and instead of the `LocationsList` component, we use the `LocationDetail` component with a single location object ❷.

From a functional perspective, however, the pages are not similar. Unlike the start page, which uses SSG, the location detail page uses SSR with `getServerSideProp` ❸. This is because as soon as we add the wish list functionality and implement the Add To/Remove button, the page's

content should change along with a user's action. Hence, we need to regenerate the HTML on each request. We discussed the differences between SSR and SSG in depth in Chapter 5.

We use the page's `context` and its `query` property to get the location ID from the dynamic URL ❹. Then we use the ID to get the matching location from the database. As before, we use the service directly instead of calling the publicly exposed API, as Next.js runs both `get...Prop` functions on the server side and can directly access the services in our application's middleware.

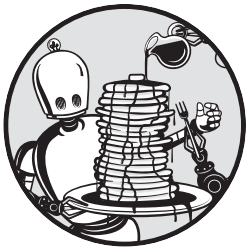
We also implement two exit scenarios. First, if there is no result, we throw an error to step into the `catch` block ❺, and by doing so, redirect the user to the *404 Not Found* error page. Otherwise, we store the first location from the results in the location property ❻ and pass it to the Next.js page function we export in the last line.

Summary

We've successfully built the frontend for the Food Finder application. At this point, you've implemented a full-stack web application that reads data from a MongoDB database and renders the results as React user interface components in Next.js. Next, we'll add an OAuth authentication flow with GitHub so that users can log in with their GitHub account and store a personalized wish list.

15

ADDING OAUTH



In this chapter, you'll add OAuth authentication to the Food Finder app, giving users the opportunity to log in with their GitHub accounts. You'll also implement the wish list page to which authenticated users can add and remove locations, as well as the button component needed to accomplish this. Lastly, you'll learn how to protect your GraphQL mutations from unauthenticated users.

Adding OAuth with next-auth

Developers usually use third-party libraries or SDKs to implement OAuth. For the Food Finder application, we'll use the *next-auth* package from Auth.js, which comes with an extensive set of preconfigured templates that allow us to connect to an OAuth service easily. These templates are called *providers*, and we'll use one of them: the GitHub provider, which adds a Log In with GitHub

button to our app. For a refresher on the OAuth authentication process, return to Chapter 9.

Creating a GitHub OAuth App

First we need to create an OAuth application using GitHub. This should give us the client ID and client secret that the Food Finder application needs to connect to GitHub. If you don't already have a GitHub account, create one now at *https://github.com*, then log in. Navigate to *https://github.com/settings/developers* and create a new OAuth app in the OAuth Apps section. Enter the Food Finder app's details in the resulting form, which should look similar to Figure 15-1.

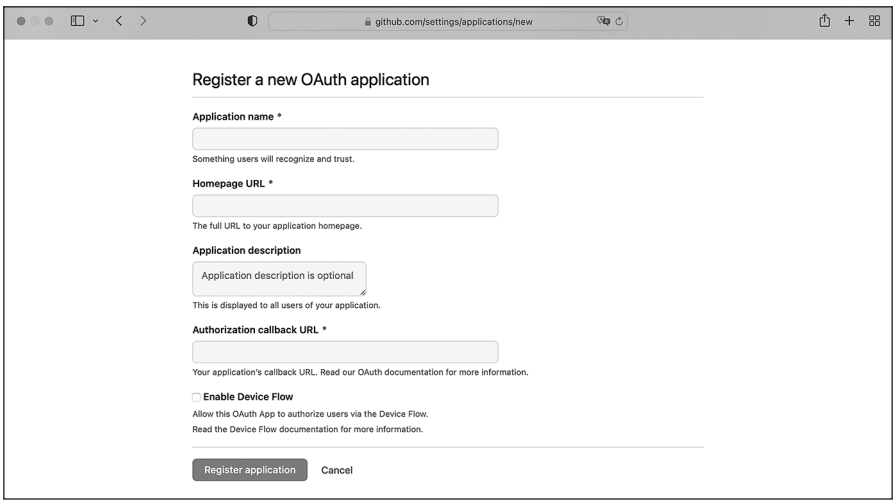


Figure 15-1: The GitHub user interface for adding a new OAuth application

Enter **Food Finder** as the name, set the home page URL to **http://localhost:3000/**, and set the authorization callback URL to **http://localhost:3000/api/auth/callback/github**. After registering the application, GitHub should show us the client ID and let us generate a client secret.

Adding the Client Credentials

Now copy these credentials as `GITHUB_CLIENT_ID` and `GITHUB_CLIENT_SECRET` to the *env.local* file in the application's code root folder. This file looks like this:

```
MONGO_URI=mongodb://backend:27017/foodfinder
GITHUB_CLIENT_ID=ADD_YOUR_CLIENT_ID_HERE
GITHUB_CLIENT_SECRET=ADD_YOUR_CLIENT_SECRET_HERE
```

Fill in the placeholders with your credentials.

Installing next-auth

To add Auth.js's OAuth SDK for *next-auth* to the Food Finder app and configure it to connect to the provider, run the following:

```
$ docker exec -it foodfinder_application npm install next-auth
```

By default, this SDK uses encrypted JWTs to store and attach session information to API requests. The library automatically handles the encryption and decryption as long as we provide it with a secret. To add such a secret, open the *env.local* file and add the following line to the end:

```
NEXTAUTH_SECRET=78f6cc4bf633b1102f4ca4d72602c60f
```

Use any secret you'd like. The string used here was randomly generated with *OpenSSL* at *https://www.usemodernfullstack.dev/api/v1/generate-secret*, and you should use a fresh one for each application.

Creating the Authentication Callback

Now we'll develop the *api/auth* route for the authorization callback URL we supplied to GitHub when registering the OAuth application. Create the *auth* folder in the *pages/api* directory containing the file *[...nextauth].ts*. The ... in the filename tells the Next.js router that this is a catch all, meaning it handles all API calls to endpoints below */auth*; for example, *auth/signin* or *auth/callback/github*. Add the code from Listing 15-1 to the file.

```
import GithubProvider from "next-auth/providers/github";
import { NextApiRequest, NextApiResponse } from "next";
import NextAuth from "next-auth";
import { createHash } from "crypto";

const createUserId = (base: string): string => {
    return createHash("sha256").update(base).digest("hex");
};

export default async function auth(req: NextApiRequest, res: NextApiResponse) {
    return await NextAuth(req, res, {
        providers: [
            GithubProvider({
                clientId: process.env.GITHUB_CLIENT_ID || "",
                clientSecret: process.env.GITHUB_CLIENT_SECRET || "",
            }),

        ],
        callbacks: {
            async jwt({ token }) {
                if (token?.email && !token.fdlst_private_userId) {
                    token.fdlst_private_userId = createUserId(token.email);
                }
                return token;
            },
```

```
            async session({ session }) {
                if (
                    session?.user?.email &&
                    !session?.user.fdlst_private_userId
                ) {
                    session.user.fdlst_private_userId = createUserId(
                        session?.user?.email
                    );
                }
                return session;
            },
        },
    });
}
```

Listing 15-1: The pages/api/auth/[...nextauth].ts *file*

We import our dependencies, including the built-in `GithubProvider` and the default `crypto` module. Then we create a simple `createUserId` function, which takes a string as an argument and calls the `crypto` module's `createHash` function to return the hashed user ID from this string.

Next, we create and export the default asynchronous `auth` function. To do so, we initialize the `NextAuth` module and add the `GithubProvider` to the `providers` array. We configure it to use the `clientId` and the `clientSecret` we stored in the environment variables.

Since we want to keep our application as simple as possible, we'll keep it stateless; hence, we use the `jwt` and `session` callbacks, which *next-auth* uses every time it creates a new session or JWT internally. In the callback, we calculate the hashed user ID from the user's email with our `createId` function (if it's not already available in the current token or session object). Finally, we store it in a private claim.

We've just created a new property, `fdlst_private_userId`, on the `user` object in the *next-auth* session. As expected, TSC warns us that this property doesn't exist on the `Session` type. We need to augment the type's interface by adjusting the *customs.d.ts* file in our application's root directory to match Listing 15-2.

```
import mongoose from "mongoose";
import { DefaultSession } from "next-auth";

declare global {
    var mongoose: mongoose;
}

declare module "next-auth" {
    interface Session {
        user: {
            fdlst_private_userId: string;
        } & DefaultSession["user"];
    }
}
```

Listing 15-2: The updated customs.d.ts *file with the augmented* `Session` *interface*

In the updated code, we import the *next-auth* package's `DefaultSession`, which defines the default session object, and then create and redeclare the `Session` interface's `user` object with the new `fdlst_private_userId` property. Because TypeScript overwrites the existing `user` object, we explicitly add it from the `DefaultSession` object. In other words, we add our new `fdlst_private_userId` property to the `Session` interface.

Sharing the Session Across Pages and Components

With the callback URL set up, we need to ensure that a user's session is shared among all Next.js pages and React components. We can use the `useContext` hook discussed in Chapter 4, which *next-auth* provides for us. In the *pages/_app.tsx* file, wrap the application in a `SessionProvider`, as shown in Listing 15-3.

```
import "../styles/globals.css";
import "../styles/layout.css";
import type { AppProps } from "next/app";
import Layout from "components/layout";
import { SessionProvider } from "next-auth/react";

export default function App({
    Component, pageProps: { session, ...pageProps } }: AppProps) {
    return (
        <SessionProvider session={session}>
            <Layout>
                <Component {...pageProps} />
            </Layout>
        </SessionProvider>
    );
}
```

Listing 15-3: The modified pages/_app.tsx *file*

We import the `SessionProvider` from the *next-auth* package and enhance the `pageProps` with the `session` object. We store the current `session` in the provider's `session` attribute, making it available throughout the Next.js application.

Before we can access the session in the frontend and middleware, we need to add the `auth-element` with the Sign In button, which will allow users to log in.

The Generic Button Component

It's time to implement the generic button component we mentioned earlier. Technically, this component will be a generic `div` element that we'll style to look like a button, with a few variations. It will serve as the Sign In/Sign Out button in the `auth-element` and the Add To/Remove From button in the location detail component. Create a new folder, *button*, in the *components* folder, adding an *index.module.css* file with the code in Listing 15-4, as well as an *index.tsx* file.

```
.root {
    align-items: center;
    border-radius: 5px;
```

```
    color: #1d1f21;
    cursor: pointer;
    display: inline-flex;
    font-weight: 500;
    height: 35px;
    letter-spacing: 0;
    margin: 0;
    overflow: hidden;
    place-content: flex-start;
    position: relative;
    white-space: nowrap;
}

.root > a,
.root > span {
    padding: 0 1rem;
    white-space: nowrap;
}

.root {
    transition: border-color 0.25s ease-in, background-color 0.25s ease-in,
        color 0.25s ease-in;
    will-change: border-color, background-color, color;
}

.root.default,
.root.default:link,
.root.default:visited {
    background-color: transparent;
    border: 1px solid transparent;
    color: #1d1f21;
}

.root.default:hover,
.root.default:active {
    background-color: transparent;
    border: 1px solid #dbd8e3;
    color: #1d1f21;
}

.root.blue,
.root.blue:link,
.root.blue:visited {
    background-color: rgba(0, 118, 255, 0.9);
    border: 1px solid rgba(0, 118, 255, 0.9);
    color: #fff;
    text-decoration: none;
}

.root.blue:hover,
.root.blue:active {
    background-color: transparent;
    border: 1px solid #1d1f21;
    color: #1d1f21;
```

```
    text-decoration: none;
}

.root.outline,
.root.outline:link,
.root.outline:visited {
    background-color: transparent;
    border: 1px solid #dbd8e3;
    color: #1d1f21;
    text-decoration: none;
}

.root.outline:hover,
.root.outline:active {
    background-color: transparent;
    border: 1px solid rgba(0, 118, 255, 0.9);
    color: rgba(0, 118, 255, 0.9);
    text-decoration: none;
}

.root.disabled,
.root.disabled:link,
.root.disabled:visited {
    background-color: transparent;
    border: 1px solid #dbd8e3;
    color: #dbd8e3;
    text-decoration: none;
}

.root.disabled:hover,
.root.disabled:active {
    background-color: transparent;
    border: 1px solid #dbd8e3;
    color: #dbd8e3;
    text-decoration: none;
}
```

Listing 15-4: The components/button/index.module.css *file*

We add styles for each of the button variations we'd like to create. All are 35 pixels tall and have rounded corners. We define a default style, a variation with a blue background and white color, and an outlined version whose background is white. In addition, we define styles to use for deactivated buttons.

With the styles in place, we can write code for the component. Copy the contents of Listing 15-5 into the component's *index.tsx* file.

```
import React from "react";
import styles from "./index.module.css";

interface PropsInterface {
    disabled?: boolean;
    children?: React.ReactNode;
    variant?: "blue" | "outline";
    clickHandler?: () => any;
}
```

```
const Button = (props: PropsInterface): JSX.Element => {
    const { children, variant, disabled, clickHandler } = props;

    const renderContent = (children: React.ReactNode) => {
        if (disabled) {
            return (
                <span className={styles.span}>
                    {children}
                </span>
            );
        } else {
            return (
                <span className={styles.span} onClick={clickHandler}>
                    {children}
                </span>
            );
        }
    };

    return (
        <div
            className={[
                styles.root,
                disabled ? styles.disabled : "",
                styles[variant || "default"],
            ].join(" ")}
        >
            {renderContent(children)}
        </div>
    );
};

export default Button;
```

Listing 15-5: The components/button/index.tsx *file*

After importing the dependencies, we define the interface for the component's `prop` argument. We also define the `Button` component as a function that returns a JSX element and then use object-destructuring syntax to split the `props` object into constants representing the object's key-value pairs. We define the internal `renderContent` function with one argument, `children`, typed as a `ReactNode` and rendered wrapped in a span element. Depending on the state of the `disabled` property, we also add the click handler from the `props` object. The component itself returns a `div` that we styled to look like a button.

The AuthElement Component

Although we've added the *next-auth* package to the project, created the OAuth API route, and configured our OAuth provider, we still can't access session information, as there is no Sign In button. Let's create this `AuthElement` component and then add it to the header. This component uses our default

button component, and as soon as the user is logged in, it displays their full name, as well as a link to their wish list.

Create the folder *auth-element* in the *components/header* directory and then add the *index.module.css* file with the code in Listing 15-6.

```
.root {
    align-items: center;
    display: flex;
    justify-content: space-between;
    margin: 0;
    padding: 1rem 0;
    width: auto;
}

.root > * {
    margin: 0 0 0 2rem;
}

.name {
    margin: 1rem 0 0 0;
}

@media (min-width: 600px) {
    .name {
        margin: 0 0 0 1rem;
    }
}
```

Listing 15-6: The components/header/auth-element/index.module.css *file*

We define a set of basic styles for the component, using a flexbox and margins to align them vertically, and change their layout for smaller screens.

To write the component itself, add an *index.tsx* file to the *auth-element* folder and enter the code from Listing 15-7 into it.

```
import Link from "next/link";
import { signIn, signOut, useSession } from "next-auth/react";
import Button from "components/button";
import styles from "./index.module.css";

const AuthElement = (): JSX.Element => {
    const { data: session, status } = useSession();

    return (
        <>
            { status === "authenticated" (
                <span className={styles.name}>
                    Hi <b>{session?.user?.name}</b>
                </span>
            )}
```

```
            <nav className={styles.root}>
                {status === "authenticated" && (
                    <>
                        <Button variant="outline">
                            <Link
href={`/list/${session?.user.fdlst_private_userId}`}
                            >
                                Your wish list
                            </Link>
                        </Button>

                        <Button variant="blue" clickHandler={() => signOut()}>
                            Sign out
                        </Button>
                    </>
                )}
                {status == "unauthenticated" && (
                    <>
                        <Button variant="blue" clickHandler={() => signIn()}>
                            Sign in
                        </Button>
                    </>
                )}
            </nav>
        </>
    );
};
export default AuthElement;
```

Listing 15-7: The components/header/auth-element/index.tsx *file*

The most notable imports are the `signIn` and `signOut` functions and the `useSession` hook from *next-auth*. The latter enables us to access session information easily, whereas the two functions trigger the sign-in flow or terminate the session.

We then define the `AuthElement` component and retrieve the `session data` and the `session status` from the `useSession` hook. We need both of these to construct the JSX element we return from the component. On the client side, we can access the session information directly via the `useSession` hook. On the server side, though, we'll need to access it through the JWT, because the session information is part of the API request's HTTP cookies.

When the session's status is authenticated, we render the user's name from the session data and add the Your Wish List and Sign Out buttons to the navigation's `nav` element. Otherwise, we add the Sign In button to start the OAuth flow. For all of those, we use the generic button component and the `signIn` and `signOut` functions we imported from the *next-auth* module, both of which handle the OAuth flow automatically.

We use the `next/link` element to link to the user's wish list. (This is another Next.js page we'll implement in a moment.) The wish list is available at the dynamic route */list/:userId*, which uses the user ID we created by hashing the user's email address and storing it in `fdlst_private_userId`.

Adding the AuthElement Component to the Header

Now we have to add the new component to the header. Open the *index.tsx* file in the *components/header* directory and adjust it so that it matches Listing 15-8.

```
import styles from "./index.module.css";
import Logo from "components/header/logo";
import AuthElement from "components/header/auth-element";
const Header = (): JSX.Element => {
    return (
        <header className={styles.root}>
            <div className="layout-grid">
                <Logo />
                <AuthElement />
            </div>
        </header>
    );
};

export default Header;
```

Listing 15-8: The modified components/header/index.tsx *file*

The update is simple; we import the `AuthElement` component and add it next to the `Logo` inside the `header`.

Test the OAuth workflow to see our session management in practice. When you open *http://localhost:3000*, the Sign In button should be in the header, as in Figure 15-2.

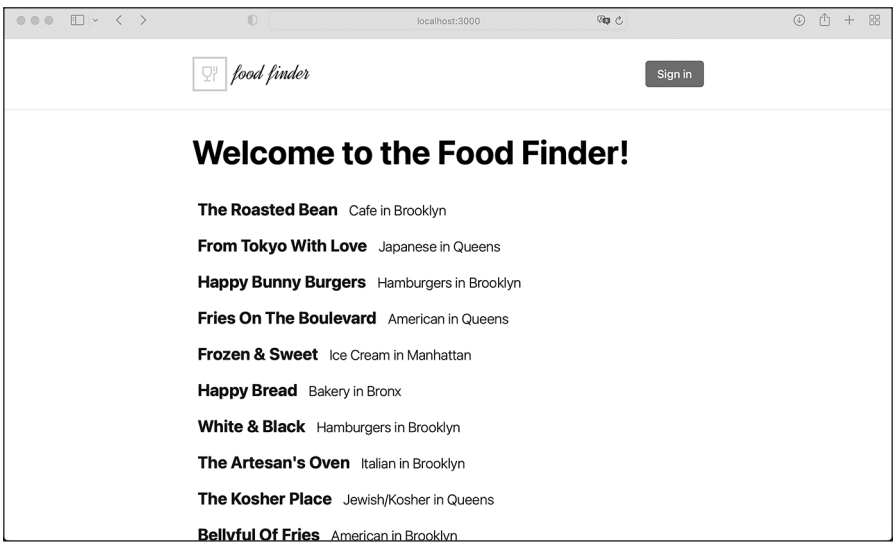


Figure 15-2: The header in a logged-out state with the Sign In button

Let's log in using OAuth. Click the **Sign In** button, and *next-auth* should redirect you to the login screen, where you can select to sign in with the configured OAuth providers to use (Figure 15-3).

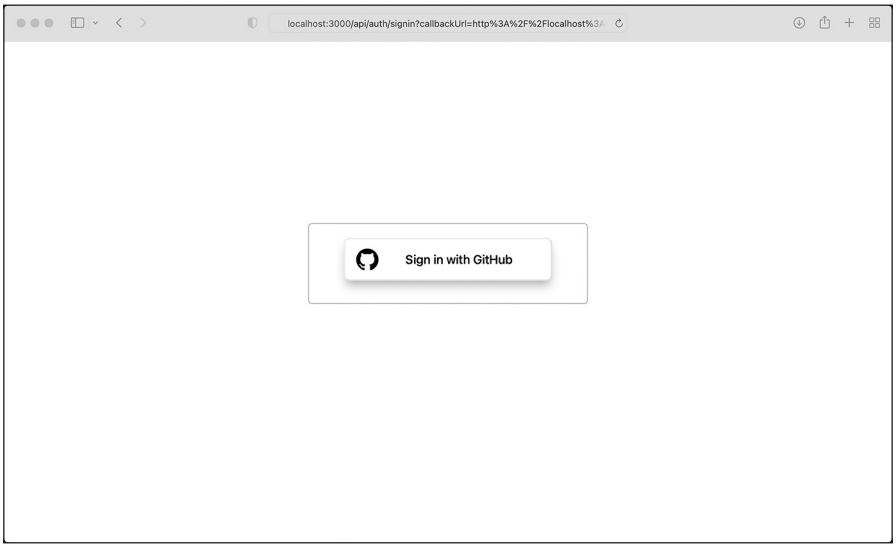


Figure 15-3: OAuth requires us to choose a provider.

Click the button to log in. OAuth should redirect you to the application, where the `AuthElement` renders your name and new buttons based on the session information. The screen should look similar to Figure 15-4.

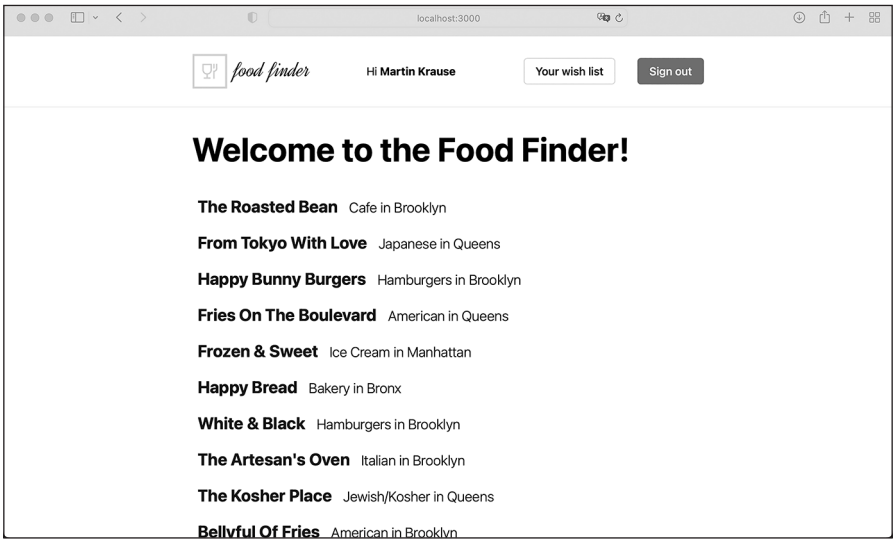


Figure 15-4: The header in the logged-in state with the session information

The header element has changed according to the session's state. We display the current user's name received from the OAuth provider, the link to their public wish list, and the Sign Out button.

The Wish List Next.js Page

The wish list button in the header should link to a wish list page at the dynamic URL *list/:userId*. This regular Next.js page should display all locations whose `on_wishlist` property contains the user ID specified in the dynamic URL. It will look quite similar to the start page, and we can build it out of existing components.

To create the page's route, create the *list* folder with the *[userId].tsx* file in the *pages* directory. Then add the code from Listing 15-9 to this *.tsx* file.

```
import type {
    GetServerSideProps,
    GetServerSidePropsContext,
    NextPage,
    PreviewData,
    InferGetServerSidePropsType,
} from "next";
import Head from "next/head";
import { ParsedUrlQuery } from "querystring";

import dbConnect from "middleware/db-connect";
import { onUserWishlist } from "mongoose/locations/services";
import { LocationType } from "mongoose/locations/schema";
import LocationsList from "components/locations-list";

import { useSession } from "next-auth/react";

const List: NextPage = (
    props: InferGetServerSidePropsType<typeof getServerSideProps>
) => {
    const locations: LocationType[] = JSON.parse(props.data?.locations);
    const userId: string | undefined = props.data?.userId;
    const { data: session } = useSession();
    let title = `The Food Finder- A personal wish list`;
    let isCurrentUsers =
        userId && session?.user.fdlst_private_userId === userId;
    return (
        <div>
            <Head>
                <title>{title}</title>
                content={`The Food Finder. A personal wish list.`}
            </Head>
            <h1>
                {isCurrentUsers ? " Your " : " A "}
                wish list!
            </h1>
            {isCurrentUsers && locations?.length === 0 && (
                <>
```

```
                    <h2>Your list is currently empty! :(</h2>
                    <p>Start adding locations to your wish list!</p>
                </>
            )}
            <LocationsList locations={locations} />
        </div>
    );
};

export const getServerSideProps: GetServerSideProps = async (
    context: GetServerSidePropsContext<ParsedUrlQuery, PreviewData>
) => {
    let { userId } = context.query;
    let locations: LocationType[] | [] = [];
    try {
        await dbConnect();
        locations = await onUserWishlist(userId as string);
    } catch (err: any) {}
    return {
        // the props will be received by the page component
        props: {
            data: { locations: JSON.stringify(locations), userId: userId },
        },
    };
};
export default List;
```

Listing 15-9: The pages/list/[userId].tsx *file*

Although we want the wish list page to look similar to the start page, we use SSR, with `getServerSideProps`, as we did for the location detail page. The wish list page is highly dynamic; hence, we need to regenerate the HTML on each request.

Another approach would be to use client-side rendering, then request the user's locations through the GraphQL API in a `useEffect` hook. However, this would cause the user to see a loading screen each time they opened the wish list page. We can avoid this inferior user experience altogether with SSR.

In the server-side part of the page's code, we first extract the URL parameter, `userId`, from the context's query object. We use the user's ID and the `onUsersWishlist` service to get all locations for the user's wish list. If there is an error, we simply continue instead of redirecting to the *404* error page, rendering an empty list.

We then pass the locations array and the user's ID to the Next.js page, where we extract the locations as usual, as well as the `userId`. We compare the user ID from the URL with the user ID in the current session. If they match, we know that the currently logged-in user has visited their own wish list and adjust the user interface accordingly.

Adding the Button to the Location Detail Component

We can now visit the wish list page, but it will always be empty. We haven't yet provided users with a way to add items to it. To change this, we'll place

a button in the location details component that lets users add or remove a particular location. We'll use the generic button component and session information. Open the *index.ts* file in the *components/location-details.tsx* directory and modify the code to match Listing 15-10.

```
import { LocationType } from "mongoose/locations/schema";
import styles from "./index.module.css";

import { useSession } from "next-auth/react";
import { useEffect, useState } from "react";
import Button from "components/button";

interface PropsInterface {
    location: LocationType;
}

interface WishlistInterface {
    locationId: string;
    userId: string;
}

const LocationDetail = (props: PropsInterface): JSX.Element => {
    let location: LocationType = props.location;

    const { data: session } = useSession();
    const [onWishlist, setOnWishlist] = useState<Boolean>(false);
    const [loading, setLoading] = useState<Boolean>(false);

    useEffect(() => {
        let userId = session?.user.fdlst_private_userId;
        setOnWishlist(
            userId && location.on_wishlist.includes(userId) ? true : false
        );
    }, [session]);

    const wishlistAction = (props: WishlistInterface) => {

        const { locationId, userId } = props;

        if (loading) { return false; }
        setLoading(true);

        let action = !onWishlist ? "addWishlist" : "removeWishlist";

        fetch("/api/graphql", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({
                query: `mutation wishlist {
                    ${action}(
                        location_id: "${locationId}",
                        user_id: "${userId}"
```

```
                ) {
                    on_wishlist
                }
            }`,
        }),
    })
    .then((result) => {
        if (result.status === 200) {
            setOnWishlist(action === "addWishlist" ? true : false);
        }
    })
    .finally(() => {
        setLoading(false);
    });
};

return (
    <div>
        {location && (
            <ul className={styles.root}>
                <li>
                    <b>Address: </b>
                    {location.address}
                </li>
                <li>
                    <b>Zipcode: </b>
                    {location.zipcode}
                </li>
                <li>
                    <b>Borough: </b>
                    {location.borough}
                </li>
                <li>
                    <b>Cuisine: </b>
                    {location.cuisine}
                </li>
                <li>
                    <b>Grade: </b>
                    {location.grade}
                </li>
            </ul>
        )}

        {session?.user.fdlst_private_userId && (
            <Button
                variant={!onWishlist ? "outline" : "blue"}
                disabled={loading ? true : false}
                clickHandler={() =>
                    wishlistAction({
                        locationId: session?.user.fdlst_private_userId,
                        userId: session?.user?.userId,
                    })
                }
```

```
                >
                    {onWishlist && <>Remove from your Wishlist</>}
                    {!onWishlist && <>Add to your Wishlist</>}
                </Button>
            )}

        </div>
    );
};
export default LocationDetail;
```

Listing 15-10: The modified components/location-details/index.tsx *file*

First we import `useSession` from `next-auth`, `useEffect` and `useState` from React, and the generic `Button` component. Then we define `WishlistInterface`, the interface for the `wishlistAction` function we'll implement in a bit.

Inside the component, we get the `session` from the `useSession` hook, then create the `onWishlist` and `loading` state variables with `useState` as Boolean values. We use the first state variable to specify whether a location is currently on the user's wish list, then update the user interface accordingly. We calculate the initial state in the `useEffect` hook based on the location's `on_wishlist` property. As soon as we've successfully added or removed the location to or from the wish list, we update the state variable and the button's text.

We implement the `wishlistAction` function to update the `on_wishlist` property. First we deconstruct the argument object and then check the `loading` state to see if there is currently a running request. If so, we exit the function. Otherwise, we set the loading state to `true` to block the user interface, calculate the action for the GraphQL mutations, and use it to call the `wishlist` mutation. After successfully modifying the document in the database, we update the `onWishlist` state and unblock the user interface.

We check the current session to see if the user is logged in. If so, we render the `Button` component and set the `disabled` and class name attributes based on the loading state, as well as an on-click event. With each click of the button, we call the `wishlistAction` function with the current location ID and user ID as arguments. Finally, we set the button's text based on the `onWishlist` state, either adding the current location to the wish list or removing it.

Try adding and removing a few locations to the wish list before moving on. Check that the button's text changes accordingly and that a list of locations similar to the one on the start page appears on the wish list page.

Securing the GraphQL Mutations

There is one more thing we have to do to wrap up the application: secure the GraphQL API. While the queries should be publicly available, the mutations should be accessible only to logged-in users, who should be able to add or remove only their own user ID for the `on_wishlist` property.

But if you test the API with the `curl` command, you'll see that, currently, everyone can access the API. Note that you must enter the values supplied to the `-d` flag on a single line, or the server might return an error:

```
$ curl -v \
    -X POST \
    -H "Accept: application/json" \
    -H "Content-Type: application/json" \
    -d '{"query":"mutation wishlist {removeWishlist(location_id: \"12340\",
 user_id: \"exampleid\") {on_wishlist}}"}' \
    http://localhost:3000/api/graphql

< HTTP/1.1 200 OK
<
{"data":{"removeWishlist":{"on_wishlist":[]}}}
```

As a test, we send a simple mutation to remove the location with the ID 12340 from a nonexistent user's wish list. (The mutation won't work, which is fine; we just want to verify whether the API is accessible to the public.) The command receives a *200* response and the expected JSON, proving that the mutations are public.

Let's implement an `authGuard` to protect our mutations. A *guard* is a pattern that checks a condition and then throws an error if it isn't met, and an *auth guard* protects a route or an API from unauthorized access.

We begin by creating the file *auth-guards.ts* in the *middleware* folder and adding the code in Listing 15-11.

```
import { GraphQLError } from "graphql/error";
import { JWT } from "next-auth/jwt";

interface paramInterface {
    user_id: string;
    location_id: string;
}

interface contextInterface {
    token: JWT;
}

export const authGuard = (
    param: paramInterface,
    context: contextInterface
): boolean | Error => {

❶ if (!context || !context.token || !context.token.fdlst_private_userId) {
        return new GraphQLError("User is not authenticated", {
            extensions: {
                http: { status: 500 },
                code: "UNAUTHENTICATED",
            },
        });
    }
```

```
❷ if (context?.token?.fdlst_private_userId !== param.user_id) {
        return new GraphQLError("User is not authorized", {
            extensions: {
                http: { status: 500 },
                code: "UNAUTHORIZED",
            },
        });
    }
    return true;
};
```

Listing 15-11: The middleware/auth-guards.ts *file*

We also import `JWT` from `next-auth` and the `GraphQLError` constructor from `graphql`. We'll use the latter to create the error objects returned to the user if authentication fails. Next, we define our interfaces for the `authGuard` function's arguments and export the function itself.

We'll call the auth guard from the mutation resolver with two parameters: an object with the user ID and the location ID, for which we defined the `paramInterface`, and the context object with the token, the `contextInterface`. The auth guard returns either a Boolean indicating that authentication succeeded or an error. In the `authGuard` function, we verify that every access to our mutation has a token with a private claim ❶ and that the user ID in the private claim matches the user ID we pass to the mutation ❷. In other words, we verify that a logged-in user has made the API request and that they're modifying their own wish list.

If the checks fail, we create an error with a message and code. In addition, we set the HTTP status code to *500*. Remember that unlike REST APIs, which rely on an extensive list of precise HTTP status codes to communicate with the caller, a GraphQL API usually uses either *200* or *500* as the status code for errors. Broadly speaking, we send a *500* status code when GraphQL can't execute the query at all and *200* when the query can be executed. In both cases, the GraphQL API should include precise information about what error occurred.

Now we must pass the user's OAuth token to the resolvers, which will then pass it to the auth guard. To do so, we'll use the context function we implemented in the `startServerAndCreateNextHandler` function, found in the *pages/api/graphql.ts* file. Open the file and adjust it to match the code in Listing 15-12.

```
import { ApolloServer, BaseContext } from "@apollo/server";
import { startServerAndCreateNextHandler } from "@as-integrations/next";

import { resolvers } from "graphql/resolvers";
import { typeDefs } from "graphql/schema";
import dbConnect from "middleware/db-connect";

import { NextApiHandler, NextApiRequest, NextApiResponse } from "next";

import { getToken } from "next-auth/jwt";
```

```
const server = new ApolloServer<BaseContext>({
    resolvers,
    typeDefs,
});

const handler = startServerAndCreateNextHandler(server, {
    context: async (req: NextApiRequest) => {
        const token = await getToken({ req });
        return { token };
    },
});

const allowCors =
    (fn: NextApiHandler) =>
    async (req: NextApiRequest, res: NextApiResponse) => {
        res.setHeader("Allow", "POST");
        res.setHeader("Access-Control-Allow-Origin", "*");
        res.setHeader("Access-Control-Allow-Methods", "POST");
        res.setHeader("Access-Control-Allow-Headers", "*");
        res.setHeader("Access-Control-Allow-Credentials", "true");

        if (req.method === "OPTIONS") {
            res.status(200).end();
        }
        return await fn(req, res);
    };

const connectDB =
    (fn: NextApiHandler) =>
    async (req: NextApiRequest, res: NextApiResponse) => {
        await dbConnect();
        return await fn(req, res);
    };

export default connectDB(allowCors(handler));
```

Listing 15-12: The modified pages/api/graphql.ts *file with the JWT token*

Unlike on the client side, where we can access the session information directly via the `useSession` hook, here we need to access it through the JWT on the server side. This is because the session information is part of the API request's HTTP cookies on the server instead of the `SessionProvider`'s shared session state, and we need to extract it from the request. To do so, we import the `getToken` function from the *next-auth* `jwt` module. Then we pass the request object we receive from the `context` function to call `getToken` and await the decoded JWT. Next, we return the token from the `context` function so that we can access the token in the resolver functions.

Finally, let's use the token to add the `authGuard` to our resolvers to protect them from unauthenticated and unauthorized access. Open the *graphql/locations/mutations.ts* file and update it with the code from Listing 15-13.

```
import { updateWishlist } from "mongoose/locations/services";
import { authGuard } from "middleware/auth-guard";
import { JWT } from "next-auth/jwt";

interface UpdateWishlistInterface {
    user_id: string;
    location_id: string;
}

interface contextInterface {
    token: JWT;
}

export const locationMutations = {
    removeWishlist: async (
        _: any,
        param: UpdateWishlistInterface,
        context: contextInterface
    ) => {

        const guard = authGuard(param, context);
        if (guard !== true) { return guard; }

        return await updateWishlist(param.location_id, param.user_id, "remove");
    },

    addWishlist: async (
        _: any,
        param: UpdateWishlistInterface,
        context: contextInterface
    ) => {

        const guard = authGuard(param, context);
        if (guard !== true) { return guard; }

        return await updateWishlist(param.location_id, param.user_id, "add");
    },
};
```

Listing 15-13: The graphql/locations/mutations.ts *file with the added* `authGuard`

We define a new interface for the context and update the context parameter to contain the JWT. Next, we add the `authGuard` function to our mutations and follow the guard pattern by returning the error immediately instead of proceeding with the code.

To test the `authGuard` functionality, run `curl` again. The command line output should look similar to Listing 15-14.

```
$ curl -v \
    -X POST \
    -H "Accept: application/json" \
    -H "Content-Type: application/json" \
    -d '{"query":"mutation wishlist {removeWishlist(location_id: \"12340\",
```

```
  user_id: \"exampleid\") {on_wishlist}}"}' \
  http://localhost:3000/api/graphql

< HTTP/1.1 500 Internal Server Error
<
{
    "errors":[
        {
            "message":"User is not authenticated",
            "locations": [ {"line":1,"column":20} ],
            "path": [ "removeWishlist" ],
            "extensions": {"code":"UNAUTHENTICATED","data":null}
        }
    ]
}
```

Listing 15-14: The `curl` command to test our API

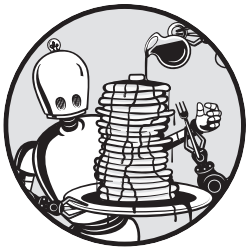
Unlike the previous `curl` call, the GraphQL API now responds with `HTTP/1.1 500 Internal Server Error` and an extensive error message, which we defined when we created the `GraphQLError` in the *auth-guards.ts* file.

Summary

We've successfully added an OAuth authentication flow to the Food Finder application. Now the user can log in with their GitHub account. Once logged in, they can maintain their personal public wish list. In addition, we've protected the GraphQL mutations, meaning they are no longer available to anyone; instead, only logged-in users can access them. In the final chapter, we'll add automated tests to evaluate the application using Jest.

16

RUNNING AUTOMATED TESTS IN DOCKER



In this short final chapter, you'll write a couple of automated tests that verify the state of the Food Finder application. Then you'll configure a Docker service to continuously run them.

We'll focus on evaluating the application's header by using a snapshot test and mocking the user session. We won't create tests for the other components or our middleware, services, or APIs. However, I encourage you to build these on your own. Try using browser-based end-to-end tests, with a specialized framework such as Cypress or Playwright, to test entire pages. You can find installation instructions and examples for both frameworks at *https://nextjs.org/docs/testing*.

Adding Jest to the Project

Install the Jest libraries with npm:

```
$ docker exec -it foodfinder-application npm install --save-dev jest \
jest-environment-jsdom @testing-library/react @testing-library/jest-dom
```

Next, configure Jest to work with our Next.js setup by creating a new file called *jest.config.js* containing the code in Listing 16-1. Save the file in the application's root folder.

```
const nextJest = require("next/jest");

const createJestConfig = nextJest({
    dir: "./",
});

const customJestConfig = {
    moduleDirectories: ["node_modules", "<rootDir>/"],
    testEnvironment: "jest-environment-jsdom",
};

module.exports = createJestConfig(customJestConfig);
```

Listing 16-1: The jest.config.js *file*

We leverage the built-in Next.js Jest configuration, so we need to configure the project's base directory to load the *config* and *.env* files into the test environment. Then we set the location of the module directories and the global test environment. We use a global setting here because our snapshot tests will require a DOM environment.

Now we want to be able to run the tests with npm commands. Therefore, add the two commands in Listing 16-2 to the `scripts` property of the project's *package.json* file.

```
    "test": "jest ",
    "testWatch": "jest --watchAll"
```

Listing 16-2: Two commands added to the package.json *file's* `scripts` *property*

The first command executes all available tests once, and the second continuously watches for file changes and then reruns the tests if it detects one.

Setting Up Docker

To run the tests using Docker, add another service to *docker-compose.yml* that uses the Node.js image. On startup, this service will run `npm run testWatch`, the command we just defined. In doing so, we'll continuously run the tests and get instant feedback about the application's state. Modify the file to match the code in Listing 16-3.

```
version: "3.0"
services:

    backend:
        container_name: foodfinder-backend
        image: mongo:latest
        restart: always
        environment:
            DB_NAME: foodfinder
            MONGO_INITDB_DATABASE: foodfinder
        ports:
            - 27017:27017
        volumes:
            - "./.docker/foodfinder-backend/seed-mongodb.js:
/docker-entrypoint-initdb.d/seed-mongodb.js"
            - mongodb_data_container:/data/db

    application:
        container_name: foodfinder-application
        image: node:lts-alpine
        working_dir: /home/node/code/foodfinder-application
        ports:
            - "3000:3000"
        volumes:
            - ./code:/home/node/code
        depends_on:
            - backend
        environment:
            - HOST=0.0.0.0
            - CHOKIDAR_USEPOLLING=true
            - CHOKIDAR_INTERVAL=100
        tty: true
        command: "npm run dev"

    jest:
        container_name: foodfinder-jest
        image: node:lts-alpine
        working_dir: /home/node/code/foodfinder-application
        volumes:
            - ./code:/home/node/code
        depends_on:
            - backend
            - application
        environment:
            - NODE_ENV=test
        tty: true
        command: "npm run testWatch"

volumes:
    mongodb_data_container:
```

Listing 16-3: The modified docker-compose.yml *file with the* jest *service*

Our small service, named *jest*, uses the official Node.js Alpine image we've used previously. We set the working directory and use the `volumes`

property to make our code available in this container as well. Unlike our application's service, however, the *jest* service sets the Node.js environment to `test` and runs the `testWatch` command.

Restart the Docker containers; the console should indicate that Jest is watching our files.

Writing Snapshot Tests for the Header Element

As in Chapter 8, create the *__tests__* folder to hold our test files in the application's root directory. Then add the *header.snapshot.test.tsx* file containing the code in Listing 16-4.

```
import { act, render } from "@testing-library/react";
import { useSession } from "next-auth/react";
import Header from "components/header";

jest.mock("next-auth/react");
describe("The Header component", () => {
    it("renders unchanged when logged out", async () => {
        (useSession as jest.Mock).mockReturnValueOnce({
            data: { user: {} },
            status: "unauthenticated",
        });
        let container: HTMLElement | undefined = undefined;
        await act(async () => {
            container = render(<Header />).container;
        });
        expect(container).toMatchSnapshot();
    });

    it("renders unchanged when logged in", async () => {
        (useSession as jest.Mock).mockReturnValueOnce({
            data: {
                user: {
                    name: "test user",
                    fdlst_private_userId: "rndmusr",
                },
            },
            status: "authenticated",
        });
        let container: HTMLElement | undefined = undefined;
        await act(async () => {
            container = render(<Header />).container;
        });
        expect(container).toMatchSnapshot();
    });
});
```

Listing 16-4: The __tests__/header.snapshot.test.tsx *file*

This test should resemble those you wrote in Chapter 8. Note that we import the `useSession` hook from *next-auth/react* and then use `jest.mock` to

replace it in the *arrange* step of each test. By replacing the session with a mocked one that returns the state, we can verify that the header component behaves as expected for both logged-in and logged-out users. We describe the test suite for the `Header` component by using the arrange, act, and assert pattern and verify that the rendered component matches the stored snapshot.

The first test case uses an empty session and the *unauthenticated* status to render the header in a logged-out state. The second test case uses a session with minimal data and sets the user's status to *authenticated*. This lets us verify that an existing session shows a different user interface than an empty session does.

If you write additional tests, make sure to add them to the *__tests__* folder.

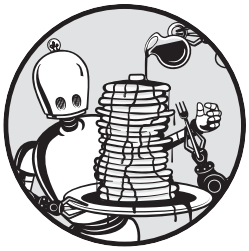
Summary

You've successfully added a few simple snapshot tests to verify that the Food Finder application works as intended. Using an additional Docker service, you can continuously verify that additional developments won't break the application.

Congratulations! You've successfully created your first full-stack application with TypeScript, React, Next.js, Mongoose, and MongoDB. You've used Docker to containerize your application and Jest to test it. With the knowledge gained in the book and its exercises, you've laid the foundation for your career as a full-stack developer.

A

TYPESCRIPT COMPILER OPTIONS



Pass any of these options to the *tsconfig.json* file's `compilerOptions` field to configure TSC's transpilation of TypeScript code to JavaScript. For more information about this process, see Chapter 3.

Here we look at the most common options. You can find more information and the complete list in the official documentation at *https://www.typescriptlang.org/tsconfig*.

`allowJs` A Boolean that specifies whether the project can import JavaScript files.

`baseUrl` A string that defines the root directory to use for resolving module paths. For example, if you set it to `"./"`, TypeScript will resolve file imports from the root directory.

`esModuleInterop` A Boolean that specifies whether TypeScript should import CommonJS, AMD, or UMD modules seamlessly or treat them differently from ES.Next modules. In general, this is necessary if you use third-party libraries without ES.Next module support.

`forceConsistentCasingInFileNames` A Boolean that specifies whether file imports are case sensitive. This can be important when some developers are working on case-sensitive filesystems and others are not, to ensure file-loading behaviors are consistent for everyone.

`incremental` A string that defines whether the TypeScript compiler should save the last compilation's project graph, use incremental type checks, and perform incremental updates on consecutive runs. This can make transpiling faster.

`isolatedModules` A Boolean that specifies whether TypeScript should issue warnings for code not compatible with third-party transpilers (such as Babel). The most common cause for those warnings is that the code uses files that are not modules; for example, they don't have any `import` or `export` statements. This value doesn't change the behavior of the actual JavaScript; it only warns about code that can't be correctly transpiled.

`jsx` A string that specifies how TypeScript handles JSX. It applies only to *.tsx* files and how the TypeScript compiler emits them; for example, the default value `react` transforms and emits the code by using `React.createElement`, whereas `preserver` does not transform the code in your component and emits it untouched.

`lib` An array that adds missing features through polyfills. In general, *polyfills* are snippets of code that add support for features and functions the target environment does not support natively. We need to emulate modern JavaScript features when we target less-compliant systems, such as older browsers or node versions. The compiler adds the polyfills defined in the `lib` array to the generated code.

`module` A string that sets the module syntax for the transpiled code. For example, if you set it to `commonjs`, TSC will transpile this project to use the legacy CommonJS module syntax with `require` for importing and `module.exports` for exporting the code, whereas with `ES2015` the transpiled code will use the `import` and `export` keywords. This is independent of the `target` property, which defines all available language features except the module syntax.

`moduleResolution` A string that specifies the module resolution strategy. This strategy also defines how TSC locates definition files for modules at compile time. Changing the approach can resolve fringe problems with the importing and exporting of modules.

`noEmit` A Boolean that defines whether TSC should produce files or only check the types in the project. Set it to `false` if you want third-party tools such as webpack, Babel.js, or Parcel to transpile the code instead of TSC.

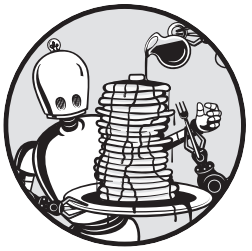
`resolveJsonModule` A Boolean that specifies whether TypeScript imports JSON files. It generates type definitions based on the JSON inside the file and validates the types on import. We need to manually enable JSON imports as TypeScript can't import them by default.

skipLibCheck A Boolean that defines whether the TypeScript compiler performs type checks on all type declaration files. Setting it to `false` decreases compilation time and is your escape hatch for working with untyped third-party dependencies.

target A string that specifies the language features to which the TypeScript code should be transpiled. For example, if you set it to `es6`, or the equivalent `ES2015`, TSC will transpile this project to ES2015-compatible JavaScript, which, for example, uses `let` and `const`.

B

THE NEXT.JS APP DIRECTORY



In version 13, Next.js introduced a new routing paradigm that uses an *app* directory instead of the *pages* directory. This appendix discusses this new feature so that you can explore it further on your own. As there are no plans to deprecate the *pages* directory, you can continue using the routing approach you learned in Chapter 5. You can even use both directories simultaneously; just be careful not to add to both directories folders and files that would create the same route, as this could cause errors.

Both the *app* and *pages* directories use folders and files to create routes. However, the *app* directory distinguishes between server and client components. In the *pages* folder, everything is a *client component*, meaning that all the code is part of the JavaScript bundle Next.js sends to the client. But

every file in the *app* directory is a *server component* by default, and its code is never sent to the client.

This appendix takes a look at the basic concepts of the new approach and then initializes a Next.js application using the new structure.

Server Components vs. Client Components

The terms *client* and *server* in this context refer to the environments in which the Next.js runtime renders a component. The client environment is the user's environment (usually the browser), whereas the server refers to the Next.js server that receives the request from the client, whether it runs on your local host or in a remote location.

With the introduction of server components, Next.js no longer purely uses client-side routing. In *server-centric* routing, the server renders components and then sends the rendered code to the client. This means the client doesn't download a routing map, which reduces the initial page size. Additionally, the user doesn't have to wait until all resources have loaded before the page becomes interactive. Next.js server components leverage React's streaming architecture to progressively render each component's content. With this model, the page becomes interactive before it has finished loading.

Server Components

Next.js server components build upon the React server components that have been available since React version 18. Because the server renders these components, they don't add anything to the JavaScript sent to the client, reducing the overall page size and increasing page performance scores. Also, the JavaScript bundle is cacheable, so the client won't redownload it when we add new additional server components, only when we add new client-side scripts through additional client components.

In addition, because these components are rendered completely on the server, they can contain sensitive server information, such as access tokens and API keys. (To add an additional layer of protection, Next.js's rendering engine replaces with an empty string all environment variables that are not explicitly prefixed with `NEXT_PUBLIC`.) Finally, we can use large-scale dependencies and additional frameworks without bloating the client-side scripts and access backend resources directly, increasing the application's performance.

Listing B-1 shows the basic structure of a server component.

```
export default async function ServerComponent(props: WeatherProps): Promise<JSX.Element> {

    return (
      <h1>The weather is {props.weather}</h1>
    );
}
```

Listing B-1: A basic server component

In Chapter 4, you learned that a React component is a JavaScript function that returns a React element; Next.js server components follow that same structure, except that they're asynchronous functions, so we can use the `async/await` pattern with `fetch`. Thus, instead of returning the React element, it returns a promise of it. The code in Listing B-1 should remind you of the `WeatherComponent` created in the previous chapters, except it doesn't contain any client-side code.

Client Components

By contrast, a client component is a component rendered by the browser rather than by the server. You already know how to write client components, because all React and Next.js components were traditionally client components.

To render these components, the client needs to receive all required scripts and their dependencies. Each component increases the bundle size, decreasing the application's performance. For that reason, Next.js offers options to optimize the application's performance, such as server-side rendering (SSR), which pre-renders the pages on the server, then lets the client add interactive elements to the page.

All components in the *app* directory are server components by default. Client components, however, can reside anywhere (for example, in the *components* directory we've used previously). Listing B-2 shows the `WeatherComponent` created in Listing 5-4 refactored into a client component that works with the *app* directory.

```
"use client";

import React, { useState, useEffect } from "react";

export default function ClientComponent (props: WeatherProps): JSX.Element {

    const [count, setCount] = useState(0);
    useEffect(() => {setCount(1);}, []);

    return (
        <h1
          onClick={() => setCount(count + 1) } >
          The weather is {props.weather},
          and the counter shows {count}
        </h1>
    );
}
```

Listing B-2: A basic client component that is similar to the `WeatherComponent` created in Listing 5-4

We export the component as the default function with the name `ClientComponent`. Because we're using the client-side hooks `useEffect` and `useState` as well as the `onClick` event handler, we need to declare the component as a client component with the `"use client"` directive at the top of the file. Otherwise, Next.js will throw an error.

Rendering Components

In Chapter 5, we performed server-side rendering with the `getServerSideProps` function and used static site generation (SSG) with the `getStaticProps` function. In the *app* directory, both functions are obsolete. If we want to optimize an application, we can instead use Next.js's built-in `fetch` API, which controls data retrieval and rendering at the component level.

Fetching Data

The new asynchronous `fetch` API extends the native fetch web API and returns a promise. Because server components are just exported functions that return a JSX element, we can declare them as asynchronous functions and then use `fetch` with the `async/await` pattern.

This pattern is beneficial because it allows us to fetch data for only the segment that uses the data rather than for an entire page. This lets us leverage React features to automatically display loading states and gracefully catch errors, as discussed in "Exploring the Project Structure" on page 269. If we follow this pattern, a loading state will block the rendering of only a particular server component and its user interface; the rest of the page will be fully functional and interactive.

NOTE *Client components shouldn't be asynchronous functions, because the way JavaScript handles asynchronous calls can easily lead to multiple re-renders and slow down the whole application. Next.js developers have discussed adding a generic* `use` *hook that lets us use asynchronous functions in client components by caching the results, but this hook is not yet finalized. If you absolutely need client-side data fetching, I recommend using a specialized library such as SWR, which you can find at* https://swr.vercel.app.

You might worry that, when each server component loads its own data, you'll end up with a massive number of requests. How do these numbers impact the overall page performance? Well, Next.js's `fetch` comes with multiple optimizations to speed up the application. For example, it automatically caches the response data for GET requests sent from a server component to the same API, reducing the number of requests.

However, POST requests aren't usually cacheable, as the data they contain might change, so `fetch` won't automatically cache them. This is a problem for us because GraphQL typically uses POST requests. Fortunately, React exposes a `cache` function that memorizes the result of the function it wraps. Listing B-3 shows an example of using `cache` with a GraphQL API.

```
import { cache } from 'react';

export const getUserFromGraphQL = cache(async (id:string) => {
    return await fetch("/graphql," { method: "POST", body: "query":"" });
});
```

Listing B-3: A simple outline of a cached POST API call

We wrap the API call in the `cache` function we imported from React and return the API's response object. Note that the cached arguments can use only primitive values because the `cache` function doesn't perform a deep comparison for the arguments.

Another optimization we can implement is to leverage the asynchronous nature of `fetch` to request data for the server component in a parallel fashion instead of sequentially. Here, the most common pattern is to use `Promise.all` to start all requests at the same time and block the rendering until all requests have been completed. Listing B-4 shows us the relevant code for this pattern.

```
const userPromiseOne = getUserFromGraphQL ("0001");
const userPromiseTwo = getUserFromGraphQL ("0002");

const [userDataOne, userDataTwo] = await Promise.all([userPromiseOne, userPromiseTwo]);
```

Listing B-4: Two parallel API calls with `Promise.all`

We set up two requests, both of which return a promise user object. Then we await the result of both promises and call `Promise.all` with an array of the previously created asynchronous API calls. The `Promise.all` function resolves as soon as both promises return their data, and then the server component's code continues.

Static Rendering

Static rendering is the default setting for both server and client components. It resembles static site generation, which we used with `getStaticProps` in Chapter 5. This rendering option pre-renders both client and server components in the server environment at build time. As a result, requests will always return the same HTML, which remains static and is never re-created.

Each component type is rendered slightly differently. For client components, the server pre-renders the HTML and JSON data; the client then receives the pre-rendered data, including the client-side script, to add interactivity to the HTML. For server components, the browser receives only the rendered payload to hydrate the component. They neither have client-side JavaScript nor use JavaScript for hydration; hence they do not send any JavaScript to the client and, in turn, don't bloat the bundled scripts.

Listing B-5 shows how to statically render the *utils/fetch-names.ts* file from Listing 5-8.

```
export default async function ServerComponentUserList(): Promise<JSX.Element> {
    const url = "https://www.usemodernfullstack.dev/api/v1/users";
    let data: responseItemType[] | [] = [];
    let names: responseItemType[] | [];
    try {
        const response = await fetch(url, { cache: "force-cache" });
        data = (await response.json()) as responseItemType[];
```

```
    } catch (err) {
        throw new Error("Failed to fetch data");
    }
    names = data.map((item) => {
        return { id: item.id, name: item.name };
    });

    return (
        <ul>
            {names.map((item) => (
                <li key="{item.id}">{item.name}</li>
            ))}
        </ul>
    );
}
```

Listing B-5: A server component that uses static rendering

First we define a server component as an asynchronous function that directly returns a `JSX.Element` wrapped in a promise.

In Chapter 5, we returned the page's data and then used the page `props` to pass it the `NextPage` function, where we generated the element. Here, after setting the `url`, we use the asynchronous `fetch` function to get the data from the remote API. Next.js will cache the results of the API call and the rendered component, and the server will reuse the generated code and never re-create it.

If you use `fetch` without an explicit cache setting, it will use `force-cache` as the default to perform static rendering. To switch to incremental static regeneration instead, replace the `fetch` call from Listing B-5 with the one in Listing B-6.

```
    const response = await fetch(url, { next: { revalidate: 20 } });
```

Listing B-6: The modified `fetch` call for ISR-like rendering

We simply add the `revalidate` property with a value of `30`. The server will then render the component statically but invalidate the current HTML 30 seconds after the first page request and re-render it.

Dynamic Rendering

Dynamic rendering replaces Next.js's traditional server-side rendering (SSR), which we used by exporting the `getServerSideProps` function from a page route in Chapter 5. Because Next.js uses static rendering by default, we must actively opt in to using dynamic rendering in one of two ways: by disabling the cache in our fetch requests or by using a dynamic function. In Listing B-7, we disable the cache.

```
export default async function ServerComponentUserList(): Promise<JSX.Element> {
    const url = "https://www.usemodernfullstack.dev/api/v1/users";
    let data: responseItemType[] | [] = [];
    let names: responseItemType[] | [];
```

```
    try {
            const response = await fetch(url, { cache: "no-cache" });
        data = (await response.json()) as responseItemType[];
    } catch (err) {
        throw new Error("Failed to fetch data");
    }
    names = data.map((item) => {
        return { id: item.id, name: item.name };
    });

    return (
        <ul>
            {names.map((item) => (
                <li key="{item.id}">{item.name}</li>
            ))}
        </ul>
    );
}
```

Listing B-7: A server component that uses dynamic rendering by disabling the cache

We explicitly set the `cache` property to `no-cache`. Now the server will refetch the data for the component upon each request.

Instead of disabling the cache, we could use dynamic functions, including the `header` function or the `cookies` function in server components and the `useSearchParams` hook in client components. These functions use dynamic data such as request headers, cookies, and search parameters that are unknown during build time and are part of the request object we pass to the function. The server needs to run these functions for each request because the required data depends on the request.

Keep in mind that dynamic rendering affects the whole route. If one server component in a route opts for dynamic rendering, Next.js will render the whole route dynamically at request time.

Exploring the Project Structure

Let's set up a new Next.js application to explore the features we've discussed. First, use the `npx create-next-app@latest` command with the `--typescript --use-npm` flags to create a sample application. When answering the setup wizard's questions, choose to use the *app* directory instead of the *pages* directory.

NOTE *You can also use the online playground at* https://codesandbox.io/s/ *to run the Next.js code examples in this appendix. Search for the official* Next.js (App router) *template when creating a new code sandbox there.*

Now enter the **`npm run dev`** command to start the application in development mode. You should see a Next.js welcome screen in your browser at *http://localhost:3000*. Unlike the welcome screen you saw in Chapter 5,

which encouraged us to edit the *pages/index.tsx* file, here the welcome screen directs us to the *app/page.tsx* file.

Take a look at the files and folders the wizard created and compare them with the ones from Chapter 5. You should see that the *pages* and *styles* directories are not part of the new structure. Instead, the router replaces both with the *app* directory. Inside it, you should see neither the *_app.tsx* file nor the *_document.tsx* file. Instead, it uses the root layout file *layout.tsx* to define the HTML wrapper for all rendered pages and the *page.tsx* file to render the root segment (the home page).

The *pages* directory uses only one file to create the final content of the page route. By contrast, the *app* directory uses multiple files to create a page route and add additional behavior.

The *page.tsx* file generates the user interface and the content for the route, and its parent folder defines the leaf segment. Without a *page.tsx* file, the URL path won't be accessible. We can then add other special files to the page's folder. Next.js will automatically apply them to this URL segment and its children. The most important of these special files are *layout.tsx*, which creates a general user interface; *loading.tsx*, which uses a React suspense boundary to automatically create a "loading" user interface while the page loads; and *error.tsx*, which uses a React error boundary to catch errors and then show the user a custom error interface.

Figure B-1 compares the files and folders for the *components/weather* page route when using the *pages* directory and the *app* directory.

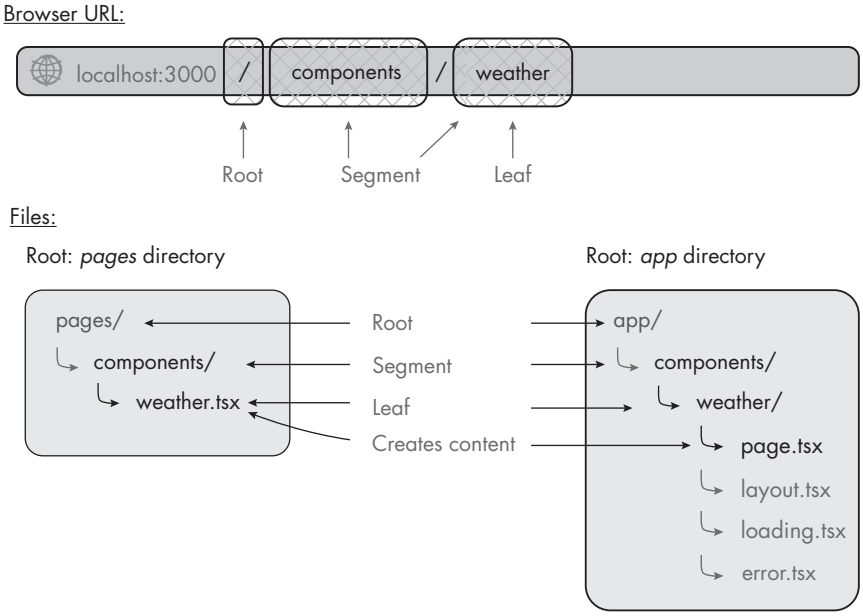


Figure B-1: Comparing the page route components/weather *in the pages and app directory structures*

When the *app* directory is the root folder, its subfolders still correspond to URL segments, but now the folder that contains the *page.tsx* file defines the URL's final leaf segment. The optional special files next to it affect only the contents of the *components/weather* page.

Let's rebuild the *components/weather* page route you created in Listing 5-1 with the *app* directory. Create the *components* folder and *weather* subfolder inside the *app* directory and then copy the *custom.d.ts* file from the previous code exercises into the root folder.

Updating the CSS

Begin by opening the existing *app/globals.css* file and replacing its content with the code from Listing B-8. We'll need to make some modifications to use special files in our component.

```
html,
body {
    background-color: rgb(230, 230, 230);
    font-family: -apple-system, BlinkMacSystemFont, Segoe UI, Roboto, Oxygen,
        Ubuntu, Cantarell, Fira Sans, Droid Sans, Helvetica Neue, sans-serif;
    margin: 0;
    padding: 0;
}

a {
    color: inherit;
    text-decoration: none;
}

* {
    box-sizing: border-box;
}

nav {
    align-items: center;
    background-color: #fff;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.25);
    display: flex;
    height: 3rem;
    justify-content: space-evenly;
    padding: 0 25%;
}

main {
    display: flex;
    justify-content: center;
}

main .content {
    height: 300px;
    padding-top: 1.5rem;
    width: 400px;
}
```

```
main .content li {
    height: 1.25rem;
    margin: 0.25rem;
}

main .loading {
    animation: 1s loading linear infinite;
    background: #ddd linear-gradient(110deg, #eeeeee 0%, #f5f5f5 15%, #eeeeee 30%);
    background-size: 200% 100%;
    min-height: 1.25rem;
    width: 90%;
}

@keyframes loading {
    to {
        background-position-x: -200%;
    }
}
main .error {
    background: #ff5656;
    color: #fff;
}

section {
    background: #fff;
    border: 1px dashed #888;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.25);
    margin: 2rem;
    padding: 0.5rem;
    position: relative;
}

section .flag {
    background: #888;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.25);
    color: #fff;
    font-family: monospace;
    left: 0;
    padding: 0.25rem;
    position: absolute;
    top: 0;
    white-space: nowrap;
}
```

Listing B-8: The app/globals.css *file with basic styles for our code examples*

We create one `nav` element for the navigation with a `main` content area below it. Then we add styles for the loading and error states we'll create later. In addition, we use the `section` element to outline the boundaries of the files and `flag` styles to add labels to the sections.

Defining a Layout

Layouts are server components that define the user interface for a particular route segment. Next.js renders this layout when this segment is active. Layouts are shared across all pages, so they can be nested into each other, and all layouts for a specific route and its children will be rendered when this route segment is active. Figure B-2 shows the relationship between the URL, the files, and the component hierarchy for the *components/weather* route.

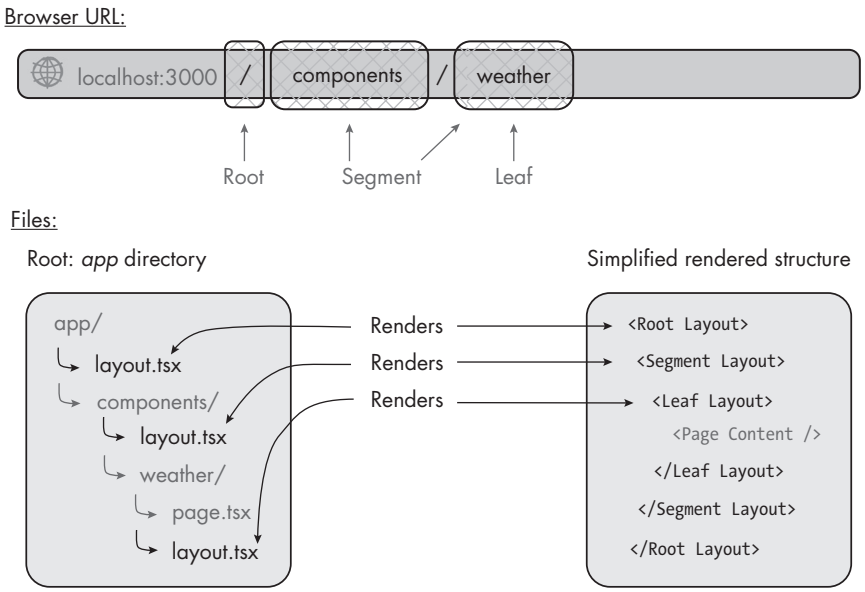


Figure B-2: The simplified layout component hierarchy

In this example, each folder contains a *layout.tsx* file. Next.js will render these in a nested fashion and make the page's content the final rendered component.

Although we can fetch data in a layout, we can't share data between a parent layout and its children. Instead, we can leverage the `fetch` API's automatic deduplication to reuse data in each child segment or component. When we navigate from one page to another, only the layouts that change are re-rendered. Shared layouts won't be re-rendered when their child segments change.

The root layout, which returns the skeleton structure with the `html` and `body` elements for the page, is required, while all other layouts we create are optional. Let's create a root layout. First, add a new interface to the end of the *custom.d.ts* file, which we copied from the previous exercise. We'll use the `LayoutProps` interface to type the layout's properties object:

```
interface LayoutProps {
    children: React.ReactNode;
}
```

Now open the *app/layout.tsx* file and replace its content with the code from Listing B-9.

```
import "./globals.css";

export const metadata = {
    title: "Appendix C",
    description: "The Example Code",
};

export default function RootLayout(props: LayoutProps): JSX.Element {
    return (
        <html lang="en">
            <body>
                <section>
                    <span className="flag">app/layout(.tsx)</span>
                    {props.children}
                </section>
            </body>
        </html>
    );
}
```

Listing B-9: The file app/layout.tsx *defines the root layout.*

We import the *global.css* file that we created earlier and then define the default SEO metadata, the page title, and the page description through the `metadata` object. This replaces the `next/head` component we used in the *pages* directory for all pages in the *app* directory.

Then we define the `RootLayout` component, which accepts an object of the `LayoutProps` type and returns a `JSX.Element`. We also create the `JSX.Element`, explicitly adding the `html` and `body` elements, then use the `section` and a `span` with the CSS class `flag` to outline the page structure. We add the `children` property from the `LayoutProps` object to wrap them with our root HTML structure.

Now let's add optional layouts to the *app/components* and *app/components/weather* folders. Create a *layout.tsx* file in each and then place the code from Listing B-10 to the *app/components/layout.tsx* file.

```
export default function ComponentsLayout(props: LayoutProps): JSX.Element {
    return (
        <section>
            <span className="flag">app/components/layout(.tsx)</span>
            <nav>Navigation Placeholder</nav>
            <main>{props.children}</main>
        </section>
    );
}
```

Listing B-10: The file app/components/layout.tsx *defines the segment layout.*

This segment layout file follows the same basic structure as the root layout. We define a layout component that receives the `LayoutProps` object with the `children` property and returns a `JSX.Element`. Unlike in the root layout, we set only the inner structure, the `nav` element with the navigation placeholder, and the `main` content area where we render the child elements from the `LayoutProps` object, representing this segment's child content (the leaf).

Lastly, create the leaf's layout by adding the code from Listing B-11 to the *app/components/weather/layout.tsx* file.

```
export default function WeatherLayout(props: LayoutProps): JSX.Element {
    return (
        <section>
            <span className="flag">app/components/weather/layout(.tsx)</span>
            {props.children}
        </section>
    );
}
```

Listing B-11: The file app/components/weather/layout.tsx *defines the leaf layout.*

The leaf's layout resembles the segment layout from Listing B-10, but it returns a more straightforward HTML structure, as the `children` property does not contain another layout; instead, it contains the page's content (in *page.tsx*), and the suspense boundary and error boundary from *loading.tsx* and *error.tsx*.

Adding the Content and Route

To expose the page route, we need to create the *page.tsx* file; otherwise, if we tried to visit the *components/weather* page route at *http://localhost:3000/components/weather*, we'd see Next.js's default *404* error page. To re-create the page content from Listing 5-1, we'll create two files. One is *component.tsx*, which contains the `WeatherComponent`, and the other is *page.tsx*, which resembles the `NextPage` wrapper we used in Listing 5-1. Of course, pages could contain additional components located in other folders.

Let's start by creating the *component.tsx* file inside the *apps/components/weather* folder and adding the code from Listing B-12 into it.

```
"use client";

import { useState, useEffect } from "react";

export default function WeatherComponent(props: WeatherProps): JSX.Element {

    const [count, setCount] = useState(0);

    useEffect(() => {
        setCount(1);
    }, []);
```

```
    return (
        <h1 onClick={() => { setCount(count + 1) }} >
            The weather is {props.weather}, and the counter shows {count}
        </h1>
    );
}
```

Listing B-12: The file app/components/weather/component.tsx *defines the* `WeatherComponent`.

This code is similar to the code in Listing 5-1 for the `WeatherComponent` constant, except we add the `"use client"` statement to explicitly set it as a client component and export it as the default function instead of storing it in a constant. The component itself has the same functionality as before: we create a headline that shows the weather string and a counter we can increase by clicking the headline.

Now we add the *page.tsx* file and the code from Listing B-13 to create the page route and expose the route to the user.

```
import WeatherComponent from "./component";

export const metadata = {
    title: "Appendix C - The Weather Component (Weather & Count)",
    description: "The Example Code For The Weather Component (Weather & Count)",
};

export default async function WeatherPage() {
    return (
        <section className="content">
            <span className="flag">app/components/weather/page(.tsx)</span>
            <WeatherComponent weather="sunny" />
        </section>
    );
}
```

Listing B-13: The file app/components/weather/page.tsx *defines the page route.*

We import the `WeatherComponent` we just created and then set the SEO metadata on the page level. Then we export the page route as the default async function. When we compare it to Listing 5-1, which contains a similar page, we see that we no longer need to export a `NextPage`; instead, we use a basic function. The *app* directory simplifies the structure of the code.

Now visit our *components/weather* page route at *http://localhost:3000/components/weather* in the browser. You should see a page that looks similar to Figure B-3.

Notice two things here. First, you should recognize the component from Chapter 5, whose counter increases when we click the headline. In addition, the combination of the styles and the `span` elements we added to each *.tsx* file visualizes the relations between the files. We see that the nested layout files resemble the simplified component hierarchy from Figure B-3.

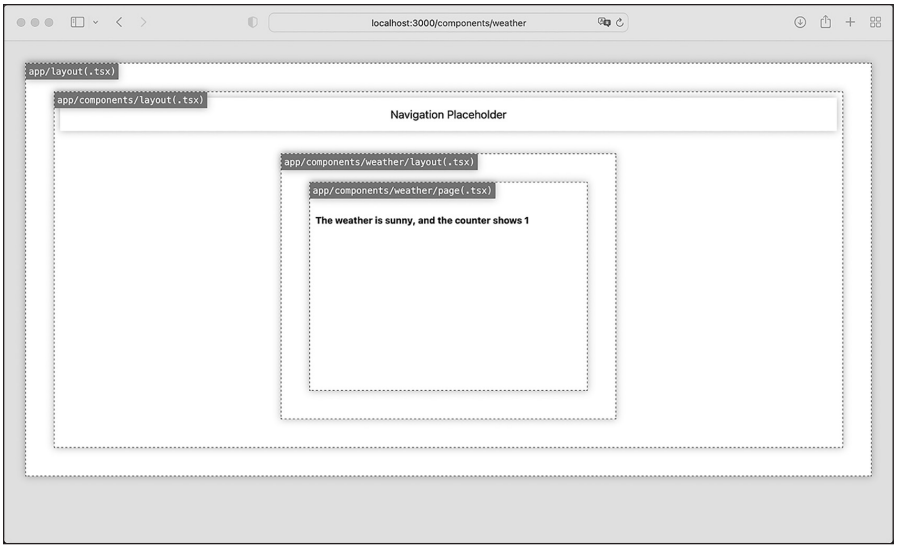


Figure B-3: The components/weather *page showing the nested components*

Catching Errors

As soon as we add an *error.tsx* file to the folder, Next.js wraps our page's content with a React error boundary. Figure B-4 shows the simplified component hierarchy of the *components/weather* route with an added *error.tsx* file.

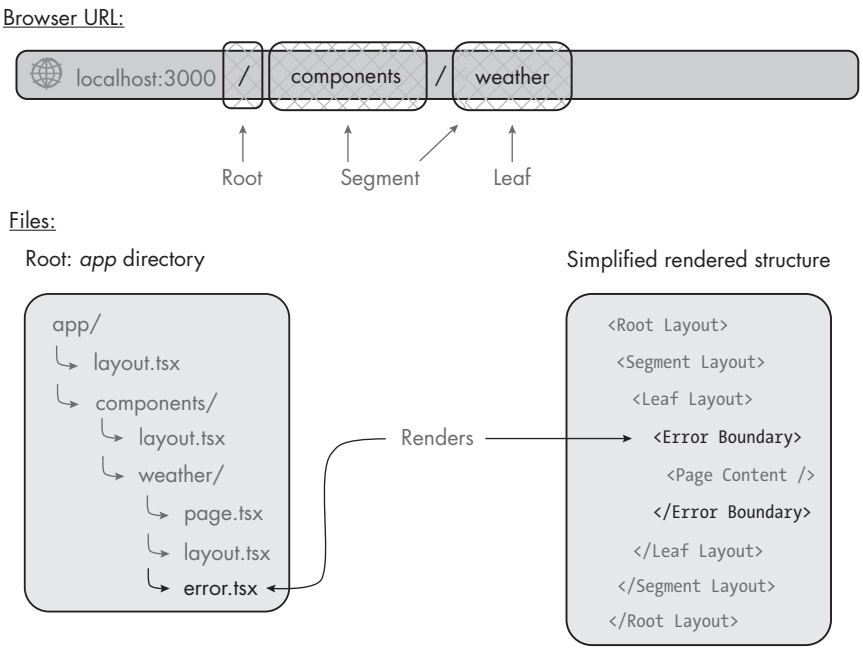


Figure B-4: The simplified layout component hierarchy includes the error boundary.

We see that the *error.tsx* file automatically creates an error boundary around the page's content. By doing so, Next.js enables us to catch errors on a page level and gracefully handle those instead of freezing the whole user interface or redirecting the user to a generic error page. Think about it as a `try...catch` block on a component level. We can now show a tailored error message and display a button that lets the user re-render the page content in a previously working state without reloading the whole application.

The *error.tsx* file exports a client component that the error boundary uses as the fallback interface. In other words, this component replaces the content when the code throws an error and activates the error boundary. As soon as it is active, it contains the error, ensuring that the layouts above the boundary remain active and maintain their internal state. The error component receives the `error` object and the `reset` function as parameters.

Let's add an error boundary to the *components/weather* route. Start by adding a new `ErrorProps` interface to type the component's properties into the *customs.d.ts* file:

```
interface ErrorProps {
    error: Error;
    reset: () => void;
}
```

Next, create the *error.tsx* file next to *page.tsx* in the *app/components/weather* directory and add the code from Listing B-14.

```
"use client";

export default function WeatherError(props: ErrorProps): JSX.Element {
    return (
        <section className="content error">
            <span className="flag">app/components/weather/error(.tsx)</span>
            <h2>Something went wrong!</h2>
            <blockquote>{props.error?.toString()}</blockquote>
            <button onClick={() => props.reset()}>Try again (re-render)</button>
        </section>
    );
}
```

Listing B-14: The file app/components/weather/error.tsx *adds the error boundary and the fallback UI.*

Because we know that the error component needs to be a client component, we add the `"use client"` directive to the top of the file and then define and export the component. We use the `ErrorProps` interface we just created to type the component's properties. We then convert the `error` property to a string and display it to inform the user of the type of error that occurred. Finally, we render a button that calls the `reset` function that the component received through the properties object. The user can re-render the component into a previous working state by clicking the button.

Now, with the error boundary in place, we'll modify *component.tsx* to throw an error if the counter hits 4 or more. Open the file and add the code from Listing B-15 below the first `useEffect` hook.

```
useEffect(() => {
    if (count && count >= 4) {
        throw new Error("Count >= 4! ");
    }
}, [count]);
```

Listing B-15: The additional `useEffect` hook for app/components/weather/component.tsx

The additional `useEffect` hook we add to the component is straightforward; as soon as the `count` variable changes, we verify the error condition, and as soon as the variable's value is 4 or more, we throw an error with the message `Count >= 4!`, which the error boundary catches and gracefully handles by showing the fallback user interface that the *error.tsx* file exports.

To test this feature, open *http://localhost:3000/components/weather* in the browser and click the headline until you trigger the error. You should see the error component instead of the weather component, as in Figure B-5.

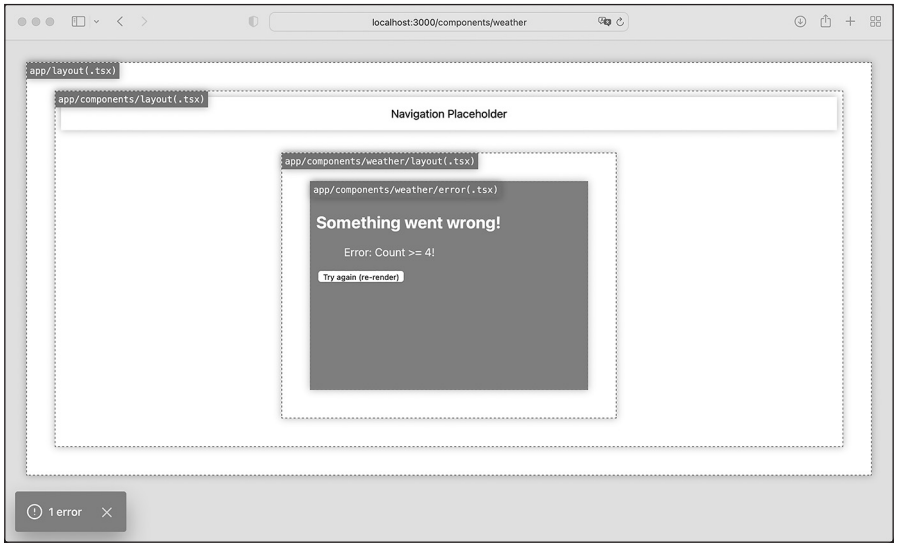


Figure B-5: The components/weather *page in the error state*

The layout markers show us that *error.tsx* has replaced *page.tsx*. We also see the string `Error: Count >=4!`, which we passed to the error constructor. As soon as we click the re-render button, *page.tsx* should replace *error.tsx*, and the screen will look like Figure B-4 previously.

Showing an Optional Loading Interface

Now we'll create the *loading.tsx* file. With this feature in place, Next.js automatically wraps the page content with a React suspense component, creating a component hierarchy that looks similar to Figure B-6.

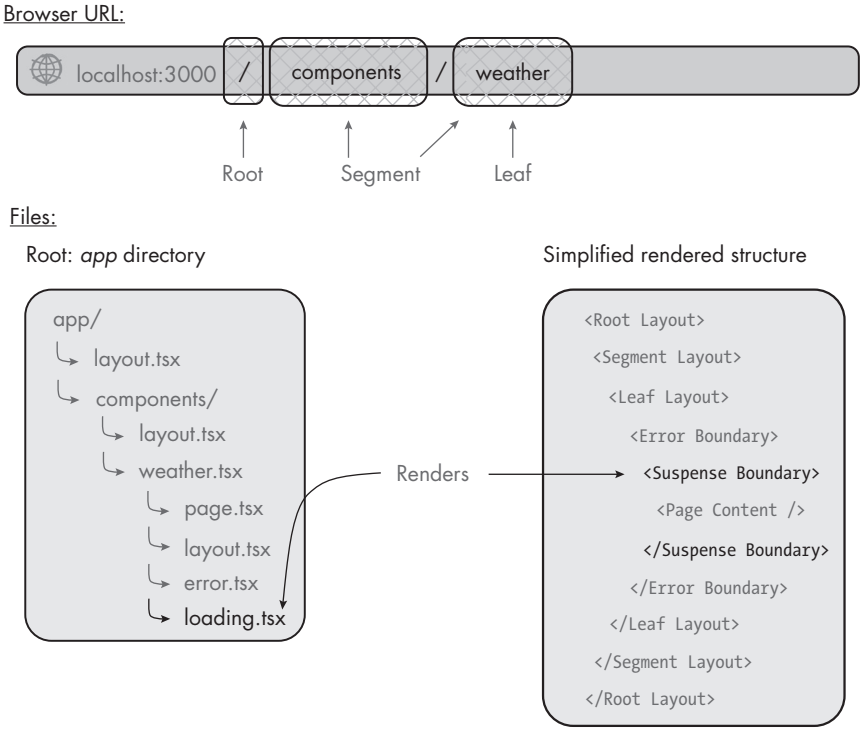


Figure B-6: The simplified layout component hierarchy with the loading interface

The *loading.tsx* file is a basic server component that returns the pre-rendered loading user interface. When we load a page or navigate between pages, Next.js will instantly display this component while loading the new segment's content. Once rendering is complete, the runtime will swap the loading state with the new content. In this way, we can easily display meaningful loading states, such as skeletons or custom animations.

Let's add a basic loading user interface to the weather component route by adding the code from Listing B-16 to the *loading.tsx* file.

```
export default function WeatherLoading(): JSX.Element {
    return (
        <section className="content">
            <span className="flag">app/components/weather/loading(.tsx)</span>
            <h1 className="loading"></h1>
        </section>
    );
}
```

Listing B-16: The file app/components/weather/loading.tsx *adds a suspense boundary with the loading user interface.*

We define and export the `WeatherLoading` component, which returns a `JSX.Element`. In the HTML, we add a headline element similar to the one in *page.tsx*, except this one adds the loading class we created in the *global.css* file to the headline and shows an animated placeholder.

When we open *http://localhost:3000/components/weather* in the browser, we should see a loading interface similar to Figure B-7.

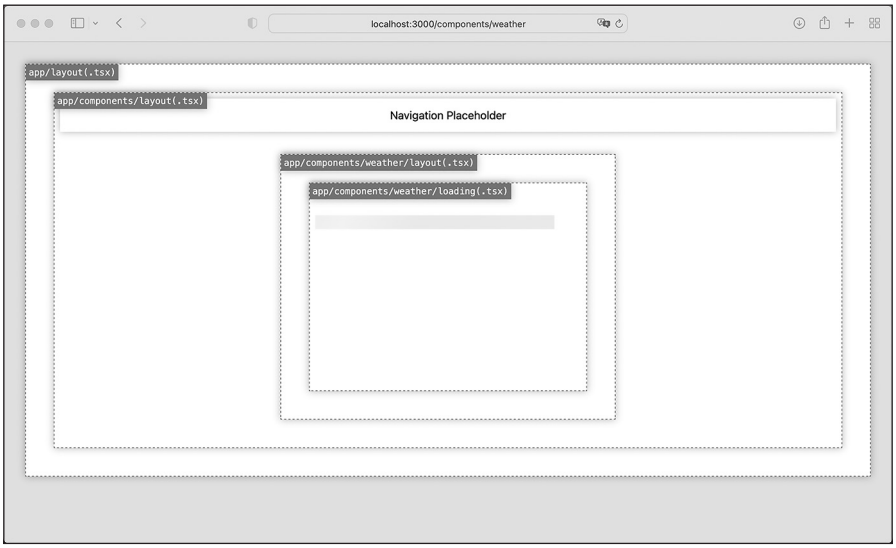


Figure B-7: The components/weather *page while loading the page's content*

If you don't see the animated placeholder, this means Next.js has already cached your segment's content.

Adding a Server Component That Fetches Remote Data

Now that you understand the folders and files in the *app* directory, let's add a server component that uses the `fetch` API to receive the list of users from the remote API *https://www.usemodernfullstack.dev/api/v1/users* and renders it to the browser. We wrote a version of this code in Chapter 5.

Create the folder *app/components/server-component* and add the special files *component.tsx*, *loading.tsx*, *error.tsx*, *layout.tsx*, and *page.tsx* to it. Then set up the component's functionality by adding the code from Listing B-17 to the *component.tsx* file.

```
export default async function ServerComponentUserList(): Promise<JSX.Element|Error> {
    const url = "https://www.usemodernfullstack.dev/api/v1/users";
    let data: responseItemType[] | [] = [];
    let names: responseItemType[] | [];
    try {
        const response = await fetch(url, { cache: "force-cache" });
        data = (await response.json()) as responseItemType[];
```

```
    } catch (err) {
        throw new Error("Failed to fetch data");
    }
    names = data.map((item) => {
        return { id: item.id, name: item.name };
    });

    return (
        <ul>
            {names.map((item) => (
                <li id="{item.id}" key="{item.id}">
                    {item.name}
                </li>
            ))}
        </ul>
    );
}
```

Listing B-17: The app/components/server-component/component.tsx *file*

Here we create a default server component that uses the `fetch` API to `await` the API response. To be able to do so, we define it as an asynchronous function that returns a promise of a `JSX.Element` or an `Error`. Then we store the API endpoint in a constant and define the variables we'll need later on. We wrap the API call in a `try...catch` statement to activate the `Error Boundary` if the API request fails. We then transform the data in a manner similarly to the way we did in Chapter 5 and return a `JSX.Element` that displays a list of users.

Now we add the loading user interface that Next.js automatically displays while we await the API's response and the component's JSX response. Place the code from Listing B-18 into the *loading.tsx* file.

```
export default function ServerComponentLoading(): JSX.Element {
    return (
        <section className="content">
            <span className="flag">
                app/components/server-component/loading(.tsx)
            </span>
            <ul id="load">
                {[...new Array(10)].map((item, i) => (
                    <li className="loading"></li>
                ))}
            </ul>
        </section>
    );
}
```

Listing B-18: The app/components/server-component/loading.tsx *file*

As before, the loading component is a server component that returns a `JSX.Element`. This time, the loading skeleton is a list with 10 items resembling the component's rendered HTML structure. You'll see that this gives the

user a good impression of the expected content and should improve the user's experience.

Next, we create the error boundary by adding the code from Listing B-19 to the *error.tsx* file.

```
"use client"; // Error components must be Client components

export default function ServerComponentError(props: ErrorProps): JSX.Element {
    return (
        <section className="content">
            <span className="flag">app/components/server-component/error(.tsx)</span>
            <h2>Something went wrong!</h2>
            <code>{props.error?.toString()}</code>
            <button onClick={() => props.reset()}>Try again (re-render)</button>
        </section>
    );
}
```

Listing B-19: The app/components/server-component/error.tsx *file*

Except for the flag outlining the file structure, the error boundary is similar to the one we used in the weather component.

Then we add the code from Listing B-20 to the *layout.tsx* file.

```
export default function ServerComponentLayout(props: LayoutProps): JSX.Element {
    return (
        <section>
            <span className="flag">app/components/server-component/layout(.tsx)</span>
            {props.children}
        </section>
    );
}
```

Listing B-20: The app/components/server-component/layout.tsx *file*

Again, the code is similar to the code we used for the weather component. We adjust only the flag outlining the component hierarchy.

Finally, with all the parts in place, we add the code from Listing B-21 to the *page.tsx* file to expose the page route.

```
import ServerComponentUserList from "./component";

export const metadata = {
    title: "Appendix C - Server Side Component (User API)",
    description: "The Example Code For A Server Side Component (User API)",
};

export default async function ServerComponentUserListPage(): JSX.Element {
    return (
        <section className="content">
            <span className="flag">app/components/server-component/page(.tsx)</span>
            {/* @ts-expect-error Async Server Component */}
```

```
            <ServerComponentUserList />
        </section>
    );
}
```

Listing B-21: The app/components/server-component/page.tsx *file*

Completing the Application with the Navigation

With two pages in the application, we can now use the `next/link` component to replace the navigation placeholder in the `nav` element. This should create a fully functional application prototype that lets us navigate between the pages. Open the *app/components/layout.tsx* file and replace the code in the file with the code from Listing B-22.

```
import Link from "next/link";

export default function ComponentsLayout(props: LayoutProps): JSX.Element {
    return (
        <section>
            <span className="flag">app/components/layout(.tsx)</span>
            <nav>
                <Link href="/components/server-component">
                    User API <br />
                    (Server Component)
                </Link>{" "}
                |
                <Link href="/components/weather">
                    Weather Component <br />
                    (Client Component)
                </Link>
            </nav>
            <main>{props.children}</main>
        </section>
    );
}
```

Listing B-22: The updated app/components/layout.tsx *file*

We import the `next/link` component and then add two links to our navigation, one pointing to the user list server component we just created and the other pointing to the weather client component.

Let's visit the application's weather component page at *http://localhost:3000/components/weather*. You should see an application that looks similar to the screenshot in Figure B-8.

As soon as you navigate between the pages, you should see the loading user interface. With the outlines we've added to all the files, we easily keep track of which files Next.js uses to render the current page.

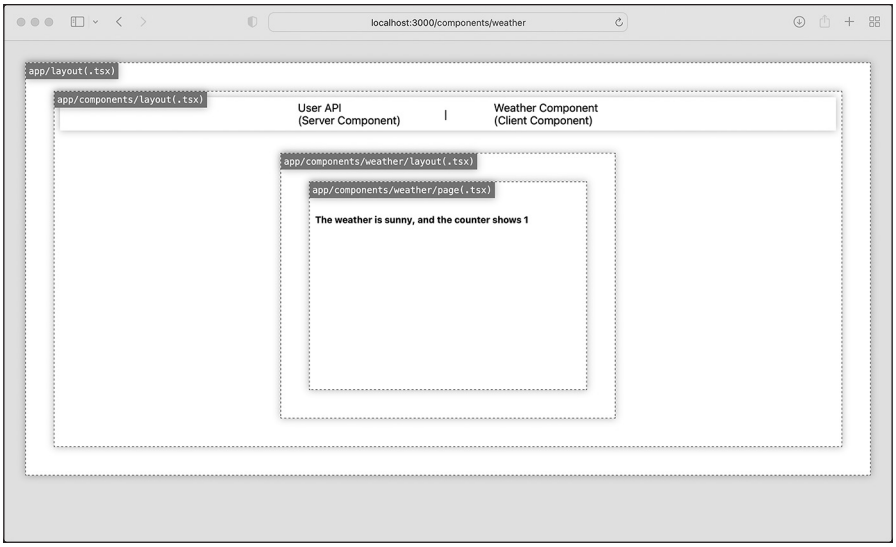


Figure B-8: The components/weather *page with the functional navigation*

Replacing API Routes with Route Handlers

If you look at the folder structure Next.js created for you, you should see that the *app* directory contains an *api* subfolder. You probably already guessed that we use this folder to define APIs. But unlike the API routes discussed in Chapter 5, which were regular functions, the *app* directory uses *route handlers*, which are functions that require a particular naming convention.

These route handlers belong in special files named *route.ts* that usually reside in a subfolder of the *app/api* folder. They are asynchronous functions that receive a `Request` object and an optional context object as parameters. We name each function after the HTTP method it should react to. For example, the code in Listing B-23 shows how to define route handlers that handle GET and POST requests for the same API.

```
import { NextRequest, NextResponse } from 'next/server';
export async function GET(request: NextRequest): Promise<NextResponse> {
    return NextResponse.json({});
}

export async function POST(request: NextRequest): Promise<NextResponse {
    return NextResponse.json({});
}
```

Listing B-23: A skeleton structure of a route.ts *file defining route handlers*

To create the route handlers, we import the `NextRequest` and `NextResponse` objects from Next.js's server package. Next.js adds additional convenience

methods for cookie handling, redirects, and rewrites. You can read more about them in the official documentation at *https://nextjs.org*.

We then define two asynchronous functions, both of which receive a `NextRequest` and return a promise of a `NextResponse`. The function names correspond to the HTTP method they should respond to. Next.js supports using the GET, POST, PUT, PATCH, DELETE, HEAD, and OPTIONS methods as function names.

When defining page routes and route handlers, remember that these files take over all requests for a given segment. This means that a *route.ts* file cannot be in the same folder as a *page.tsx* file. Also, route handlers are the only way to define APIs if you use the *app* directory: you can't have an *api* folder in both the *pages* directory and *app* directory.

Next.js has statically and dynamically evaluated route handlers. Statically evaluated route handlers will be cached and reused for every request, whereas dynamically evaluated route handlers must request the data upon each request. By default, the runtime statically evaluates all GET route handlers that don't use a dynamic function or the `Response` object. As soon as we use a different HTTP method, the dynamic `cookies` or `headers` function, or the `Response` object, the route handler becomes dynamically evaluated. The same applies to APIs with dynamic segments, which receive the dynamic parameters through the `context` object.

Let's re-create the API route *api/v1/weather/[zipcode].ts* from Listing 5-1 as a route handler that we can use in the *app* directory. Add the code from Listing B-24 to a *route.ts* file in the folder structure *app/api/v1/weather/[zipcode]*.

```
import { NextResponse, NextRequest } from "next/server";

interface ReqContext {
    params: {
        zipcode: number;
    }
}

export async function GET( req: NextRequest, context: ReqContext ): Promise<NextResponse> {
    return NextResponse.json(
        {
            zipcode: context.params.zipcode,
            weather: "sunny",
            temp: 35,
        },
        { status: 200 }
    );
}
```

Listing B-24: The route handler in app/api/v1/weather/[zipcode]/route.ts

Notice that we've used the square brackets pattern on the folder structure to access the dynamic segment through the function's second parameter `context` object.

Within the file, we import the Next.js `Response` and `NextRequest` objects from the server package and then define the interfaces for the route handler. On the `RequestContext` interface, we add the `zipcode` property to `params`, representing the API's dynamic segment. Finally, we export the asynchronous GET function, the API route handler that reacts to all GET requests for this API endpoint. It receives the request object and the request context as parameters and uses the `NextResponse`'s `json` function to return the response data. We access the URL parameter `zipcode` through the `context` object's `params` object and then add it to the response data. We set additional response options through the `json` function's second parameter, explicitly setting the HTTP status code to *200*.

Now try querying the API with `curl`:

```
$ curl -i \
    -X GET \
    -H "Accept: application/json" \
    http://localhost:3000/api/v1/weather/12345
```

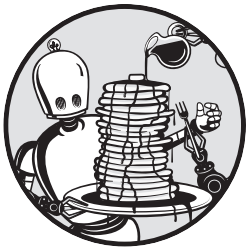
You should receive this JSON response:

```
HTTP/1.1 200 OK
content-type: application/json
--snip--
{"zipcode":"12345","weather":"sunny","temp":35}
```

This is the same response received in Chapter 5, where we accessed the API through the browser.

C

COMMON MATCHERS



In Jest, *matchers* let us check a specific condition, such as whether two values are equal or whether an HTML element exists in the current DOM. Jest comes with a set of built-in matchers. In addition, the *JEST-DOM* package from the testing library provides DOM-specific matchers.

Built-in Matchers

This section covers the most common built-in Jest matchers. You can find a complete list in the official JEST documentation at *https://jestjs.io/docs/expect*.

`toBe` This matcher is the simplest and by far the most common. It's a simple equality check to determine whether two values are identical. It behaves similarly to the strict equality (`===`) operator, as it considers type differences. Unlike the strict equality operator, however, it considers `+0` and `-0` to be different.

```
test('toBe',  () => {
    expect(1+1).toBe(2);
})
```

toEqual We use `toEqual` to perform a deep-equality check between objects and arrays, comparing all of their properties or items. This matcher ignores undefined values and items. Furthermore, it does not check the object's types (for example, whether they are instances or children of the same class or parent object). If you require such a check, consider using the `toStrictEqual` matcher instead.

```
test('toEqual', () => {
    expect([undefined, 1]).toEqual([1]);
})
```

toStrictEqual The `toStrictEqual` matcher performs a structure and type comparison for objects and arrays; passing this test requires that the objects are of the same type. In addition, the matcher considers `undefined` values and `undefined` array items.

```
test('toStrictEqual', () => {
    expect([undefined, 1]).toStrictEqual([undefined, 1]);
})
```

toBeCloseTo For floating-point numbers, we use `toBeCloseTo` instead of `toBe`. This is because JavaScript's internal calculations of floating-point numbers are flawed, and this matcher considers those rounding errors.

```
test('toBeCloseTo', () => {
    expect(0.1+0.2).toBeCloseTo(0.3);
})
```

toBeGreaterThan/toBeGreaterThanOrEqual For numeric values, we use these matchers to verify that the result is greater than or equal to a value, similar to the `>` and `>=` operators.

```
test('toBeGreaterThan', () => {
    expect(1+1).toBeGreaterThan(1);
})
```

toBeLessThan/toBeLessThanOrEqual These are the opposite of the `GreaterThan...` matchers for numeric values, similar to the `<` and `<=` operators.

```
test('toBeLessThan', () => {
    expect(1+1).toBeLessThan(3);
})
```

toBeTruthy/toBeFalsy These matchers check if a value exists, regardless of its value. They consider the six JavaScript values `0`, `' '`, `null`, `undefined`, `NaN`, and `false` to be falsy and everything else to be truthy.

```
test('toBeTruthy', () => {
    expect(1+1).toBeTruthy();
})
```

toMatch This matcher accepts a string or a regular expression, then checks if a value contains the given string or if the regular expression returns the given result.

```
test('toMatch, () => {
    expect('apples and oranges').toMatch('apples');
})
```

toContain The `toContain` matcher is similar to `toMatch`, but it accepts either an array or a string and checks these for a given string value. When used on an array, the matcher verifies that the array contains the given string.

```
test('toMatch, () => {
    expect(['apples', 'oranges']).toContain('apples');
})
```

toThrow This matcher verifies that a function throws an error. The function being checked requires a wrapping function or the assertion will fail. We can pass it a string or a regular expression, similar to the `toMatch` function.

```
function functionThatThrows() {
    throw new Error();
}

test('toThrow', () => {
    expect(() => functionThatThrows()).toThrow();
})
```

The JEST-DOM Matchers

The *JEST-DOM* package provides matchers to work directly with the DOM, allowing us to easily write tests that run assertions on the DOM, such as checking for an element's presence, HTML contents, CSS classes, or attributes.

Say we want to check that our logo element has the class name `center`. Instead of manually checking for the presence of an element and then checking its class name attribute with `toMatch`, we can use the `toHaveClass` matcher, as shown in Listing C-1.

```
<img data-testid="image" class="center full" alt="The Logo" src="logo.svg" />

test('toHaveClass', () => {
    const element = getByTestId('image');
    expect(element).toHaveClass('center');
})
```

Listing C-1: The basic syntax for testing with the DOM

First we add the data attribute `testid` to our image element. Then, in the test, we get the element using this ID and store the reference in a constant. Finally, we use the `toHaveClass` matcher on the element's reference to see if the element's class names contain the class `center`.

Let's take a look at the most common DOM-related matchers.

`getByTestId` This matcher lets us directly access a DOM element and store a reference to it, which we then use with custom matchers to assert things about this element.

```
<img data-testid="image" class="center full" alt="The Logo" src="logo.svg" />

test('toHaveClass', () => {
    const element = getByTestId('image');
--snip--
})
```

`toBeInTheDocument` This matcher verifies that an element was added to the document tree. This matcher works only on elements that are currently part of the DOM and ignores detached elements.

```
<img data-testid="image" class="center full" alt="The Logo" src="logo.svg" />

test('toHaveClass', () => {
    const element = getByTestId('image');
    expect(element).toBeInTheDocument();
})
```

toContainElement This matcher tests our assumptions about the element's child elements, letting us verify, for example, whether an element is a descendant of the first.

```
<div data-testid="parent">
    <img data-testid="image" class="center full" alt="The Logo" src="logo.svg" />
</div>

test('toHaveClass', () => {
    const parent = getByTestId('parent');
    const element = getByTestId('image');
    expect(parent).toContainElement(element);
})
```

toHaveAttribute This matcher lets us run assertions on the element's attributes, such as an image's `alt` attribute and the `checked`, `disabled`, or `error` state of form elements.

```
<img data-testid="image" class="center full" alt="The Logo" src="logo.svg" />

test('toHaveClass', () => {
    const element = getByTestId('image');
    expect(element).toHaveAttribute('alt', 'The Logo');
})
```

toHaveClass The `toHaveClass` matcher is a specific variant of the `toHaveAttribute` matcher. It lets us explicitly assert that an element has a particular class name, allowing us to write clean tests.

```
<img data-testid="image" class="center full" alt="The Logo" src="logo.svg" />

test('toHaveClass', () => {
    const element = getByTestId('image');
    expect(element).toHaveClass('center');
})
```

INDEX

B

C

D

E

F

G

H

I

J

K

L

M

N

O

P

Q

R

S

T

U

V

W

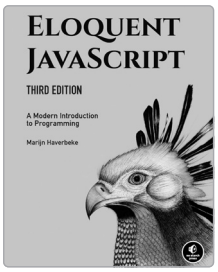
Y

The Complete Developer is set in New Baskerville, Futura, Dogma, and TheSansMono Condensed.

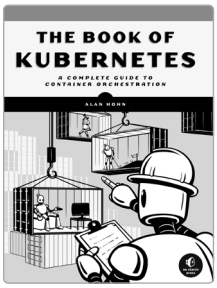
RESOURCES

Visit *https://nostarch.com/complete-developer* for errata and more information.

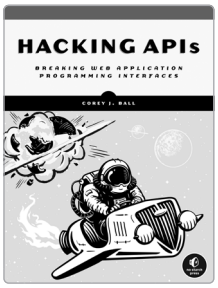
More no-nonsense books from **NO STARCH PRESS**



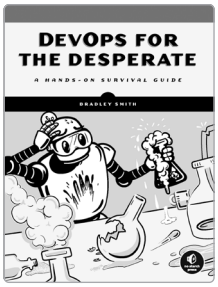
ELOQUENT JAVASCRIPT, 3RD EDITION
A Modern Introduction to Programming
BY MARIJN HAVERBEKE
472 PP., $39.99
ISBN 978-1-59327-950-9



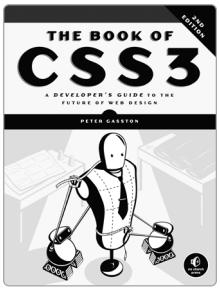
THE BOOK OF KUBERNETES
A Complete Guide to Container Orchestration
BY ALAN HOHN
384 PP., $59.99
ISBN 978-1-7185-0264-2



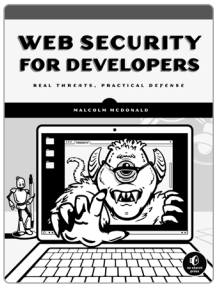
HACKING APIS
Breaking Web Application Programming Interfaces
BY COREY J. BALL
368 PP., $59.99
ISBN 978-1-7185-0244-4



DEVOPS FOR THE DESPERATE
A Hands-On Survival Guide
BY BRADLEY SMITH
176 PP., $29.99
ISBN 978-1-7185-0248-2



THE BOOK OF CSS3, 2ND EDITION
A Developer's Guide to the Future of Web Design
BY PETER GASSTON
304 PP., $34.95
ISBN 978-1-59327-580-8



WEB SECURITY FOR DEVELOPERS
Real Threats, Practical Defense
BY MALCOLM MCDONALD
216 PP., $29.95
ISBN 978-1-59327-994-3

PHONE:
800.420.7240 or
415.863.9900

EMAIL:
sales@nostarch.com

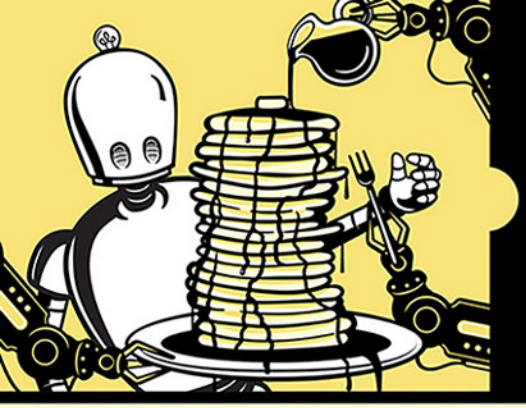
WEB:
www.nostarch.com

Never before has the world relied so heavily on the Internet to stay connected and informed. That makes the Electronic Frontier Foundation's mission—to ensure that technology supports freedom, justice, and innovation for all people—more urgent than ever.

For over 30 years, EFF has fought for tech users through activism, in the courts, and by developing software to overcome obstacles to your privacy, security, and free expression. This dedication empowers all of us through darkness. With your help we can navigate toward a brighter digital future.

ELECTRONIC FRONTIER FOUNDATION

LEARN MORE AND JOIN EFF AT EFF.ORG/NO-STARCH-PRESS



Build Full-Stack Web Applications from Scratch

Whether you've been in the developer kitchen for decades or are just taking the plunge to do it yourself, *The Complete Developer* will show you how to build and implement every component of a modern stack—from scratch.

You'll go from a React-driven frontend to a fully fleshed-out backend with Mongoose, MongoDB, and a complete set of REST and GraphQL APIs, and back again through the whole Next.js stack.

The book's easy-to-follow, step-by-step recipes will teach you how to build a web server with Express.js, create custom API routes, deploy applications via self-contained microservices, and add a reactive, component-based UI. You'll leverage command line tools and full-stack frameworks to build an application whose no-effort user management rides on GitHub logins.

You'll also learn how to:

- Work with modern JavaScript syntax, TypeScript, and the Next.js framework
- Simplify UI development with the React library
- Extend your application with REST and GraphQL APIs
- Manage your data with the MongoDB NoSQL database
- Use OAuth to simplify user management, authentication, and authorization
- Automate testing with Jest, test-driven development, stubs, mocks, and fakes

Whether you're an experienced software engineer or new to DIY web development, *The Complete Developer* will teach you to succeed with the modern full stack. After all, control matters.

Covers: Docker, Express.js, JavaScript, Jest, MongoDB, Mongoose, Next.js, Node.js, OAuth, React, REST and GraphQL APIs, and TypeScript

ABOUT THE AUTHOR

Martin Krause has been building websites from scratch for over 20 years. He has been an engineering manager at Publicis Sapient and a senior frontend architect at Razorfish, creating cutting-edge microsites and leading teams on large-scale projects.

no starch press®

THE FINEST IN GEEK ENTERTAINMENT™

nostarch.com